

Nivell 1: Entorn i Ingesta Híbrida (Code-First)

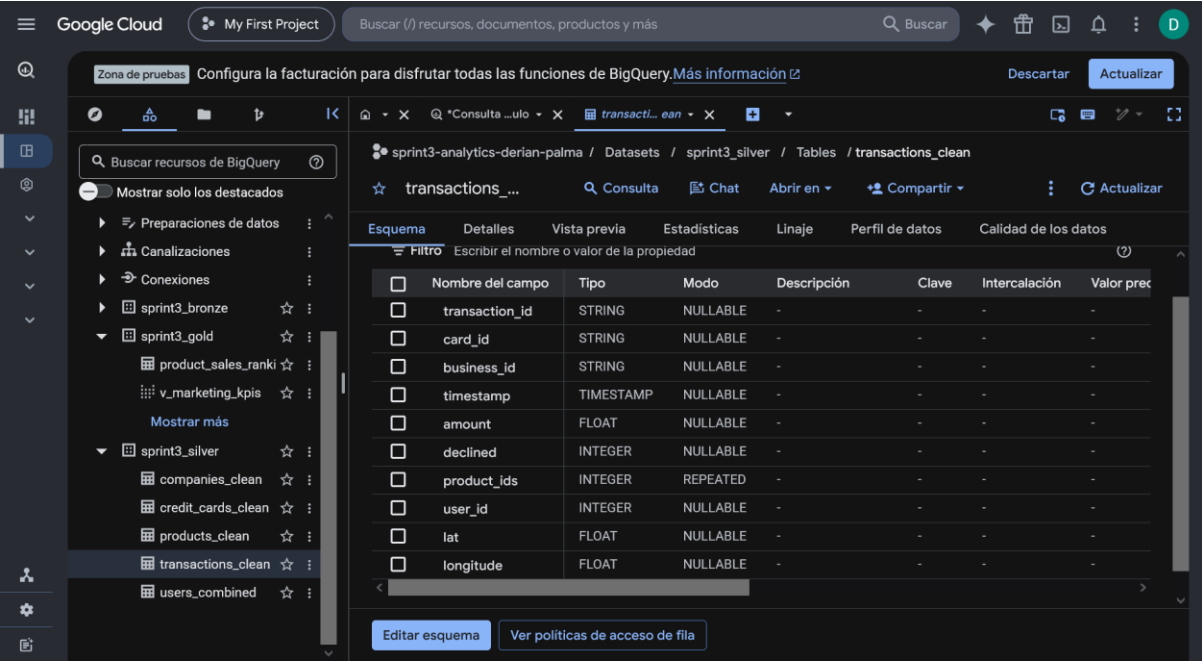
Exercici 1: Consulta sobre Taula no Optimitzada (Diagnòstic)

El Country Manager d'**Alemanya** necessita revisar urgentment les transaccions del dia **12 de març de 2022**.

Tasca:

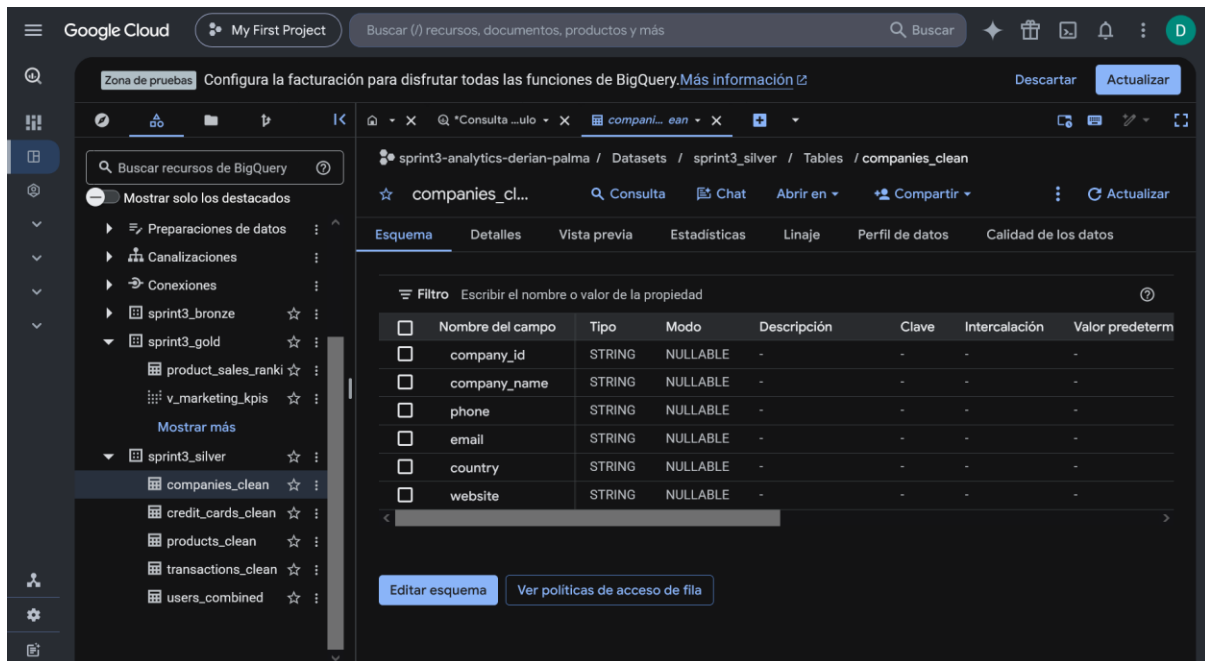
- » 1. Escribe la consulta que uneix (**JOIN**) transaccions i companyies.
- » 2. Filtra els resultats per la data indicada i el país "Germany".
- » 3. Sense executar la consulta, realitza un "Dry Run" (auditoria de costos).
🔗 Observació: Fixa't que BigQuery llegeix gairebé tota la taula tot i demanar només un dia (Full Table Scan).

Antes de elaborar la consulta, se revisaron los esquemas de transactions_clean y companies_clean para comprobar los nombres y tipos de datos procedentes del Sprint 3. Se confirmó que timestamp es de tipo TIMESTAMP y que business_id y company_id son de tipo STRING, lo que permite realizar el JOIN directamente, sin conversiones de tipo. También se verificó la existencia del campo country para filtrar por país.



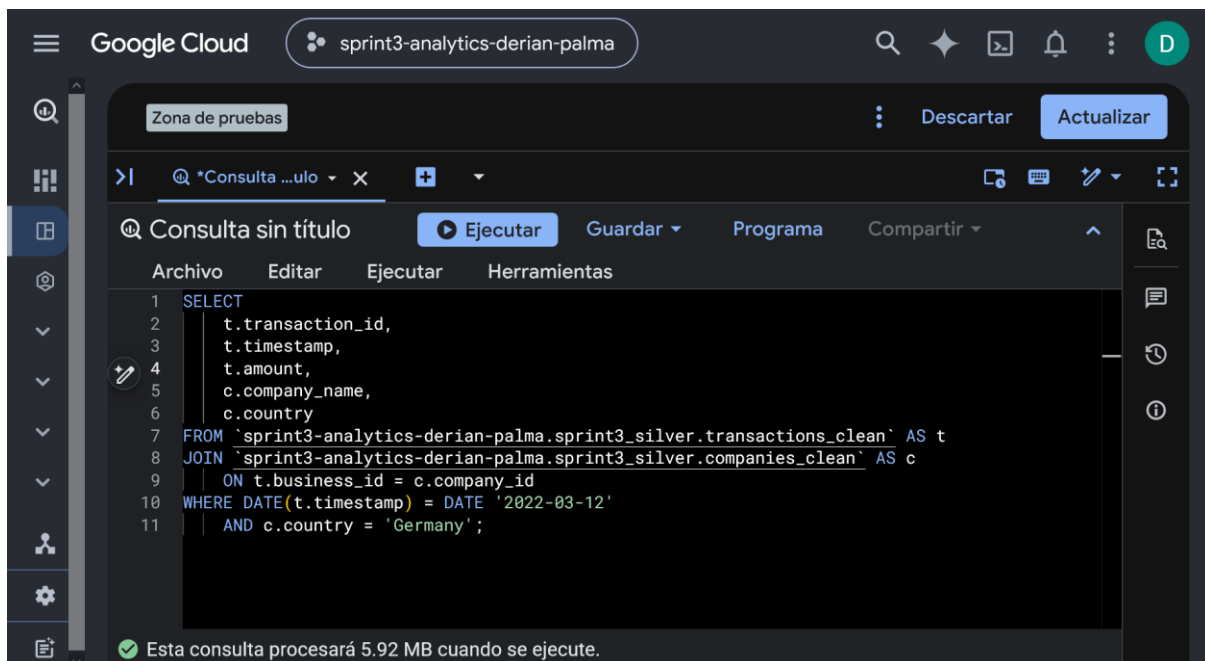
The screenshot shows the Google Cloud BigQuery interface. On the left, a sidebar lists datasets and tables, including 'transactions_clean'. The main panel displays the schema of 'transactions_clean' with columns: transaction_id, card_id, business_id, timestamp, amount, declined, product_ids, user_id, lat, and longitude. The 'timestamp' column is highlighted.

Nombre del campo	Tipo	Modo	Descripción	Clave	Intercalación	Valor pred
transaction_id	STRING	NULLABLE	-	-	-	-
card_id	STRING	NULLABLE	-	-	-	-
business_id	STRING	NULLABLE	-	-	-	-
timestamp	TIMESTAMP	NULLABLE	-	-	-	-
amount	FLOAT	NULLABLE	-	-	-	-
declined	INTEGER	NULLABLE	-	-	-	-
product_ids	INTEGER	REPEATED	-	-	-	-
user_id	INTEGER	NULLABLE	-	-	-	-
lat	FLOAT	NULLABLE	-	-	-	-
longitude	FLOAT	NULLABLE	-	-	-	-



Se ha elaborado una consulta que une `transactions_clean` y `companies_clean` mediante los campos `business_id` y `company_id`. Se filtran las transacciones del 12 de marzo de 2022 correspondientes a empresas de Alemania, mostrando el identificador de transacción, la fecha, el importe, el nombre de empresa y el país.

Sin ejecutar la consulta, el validador de BigQuery confirma que es válida y estima un procesamiento de 5,92 MB. Este valor queda registrado como diagnóstico inicial de la consulta sobre las tablas sin optimizar.



Exercici 2: Re-arquitectura i Optimització de l'Emmagatzematge (Partition & Cluster)

Escenari:

Després del diagnòstic de l'exercici anterior, hem confirmat que filtrar la taula de transaccions per data o per empresa provoca un *Full Table Scan* (lectura completa de la taula), cosa que és ineficient. Per solucionar-ho a la capa de consum (*Gold*), materialitzarem una versió optimitzada de la taula de fets.

Problema del sandbox: com que utilitzem un entorn gratuït sense targeta de crèdit, BigQuery esborra automàticament les dades de les particions amb més de 60 dies d'antiguitat. Atès que les nostres dades històriques són antigues, si particionem directament la taula, aquesta acabarà quedant buida. Per evitar-ho, ho resoldrem en dos passos: primer, actualitzarem automàticament les dates de les dades i, després, aplicarem l'optimització física.

Tasca:

Pas 1: Generació de Dades Recents (Mocking Data)

Crea una taula intermèdia anomenada `sprint3_silver.transactions_recent` a partir de la taula `sprint3_silver.transactions_clean`. El teu objectiu és mantenir totes les columnes, però substituir el timestamp original per un de nou, generat aleatòriament perquè caigui dins dels últims 50 dies.

» **Pista Tècnica:** Per fer-ho sense escriure totes les columnes a mà, pots utilitzar `SELECT * EXCEPT(timestamp)`. Per calcular la data aleatòria, hauràs de restar una quantitat de dies a l'instant actual. Et seran molt útils les funcions `TIMESTAMP_SUB()`, `CURRENT_TIMESTAMP()`, i la combinació `CAST(RAND() * 50 AS INT64)` per generar els dies de desfasament.

Pas 2: Creació de la Taula Optimitzada (Partitioning & Clustering)

Ara, crea la teva taula definitiva `sprint3_gold.fact_transactions_optimized` a partir de les dades recents que acabes de generar a `transactions_recent`. Has de construir la sentència DDL per configurar la taula amb les següents estratègies físiques d'emmagatzematge:

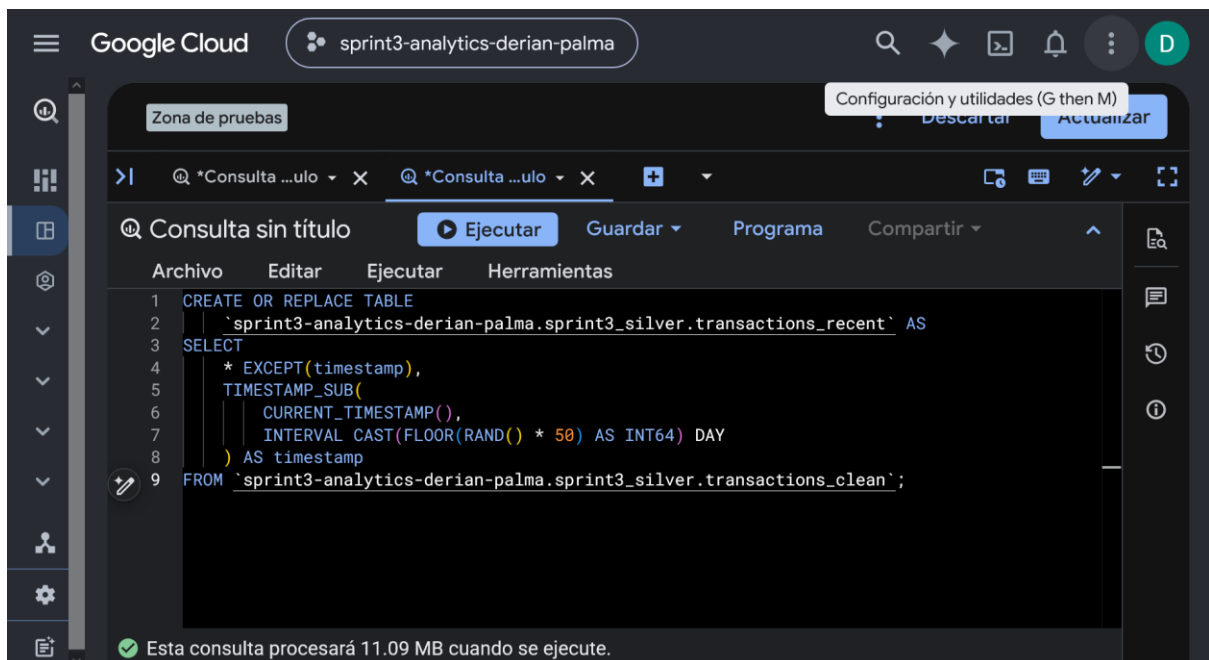
» **Particionament (Partitioning)**: Divideix la taula per la data del camp DATE(timestamp). Això permetrà que les consultes que filtrin per dia (**WHERE date = ...**) només llegeixin la partició necessària.

» **Clustering**: Ordena les dades dins de cada partició per business_id. Això accelerarà dràsticament els filtres per companyia i els encreuaments (**JOINS**) amb la dimensió d'empreses.

👉 Entrega:

- » El codi SQL utilitzat per al Pas 1.
- » El codi DDL (CREATE OR REPLACE TABLE...) utilitzat al Pas 2.
- » Una captura de pantalla dels "Detalls" de la nova taula fact_transactions_optimized on es pugui comprovar que està particionada i agrupada en clústers correctament.

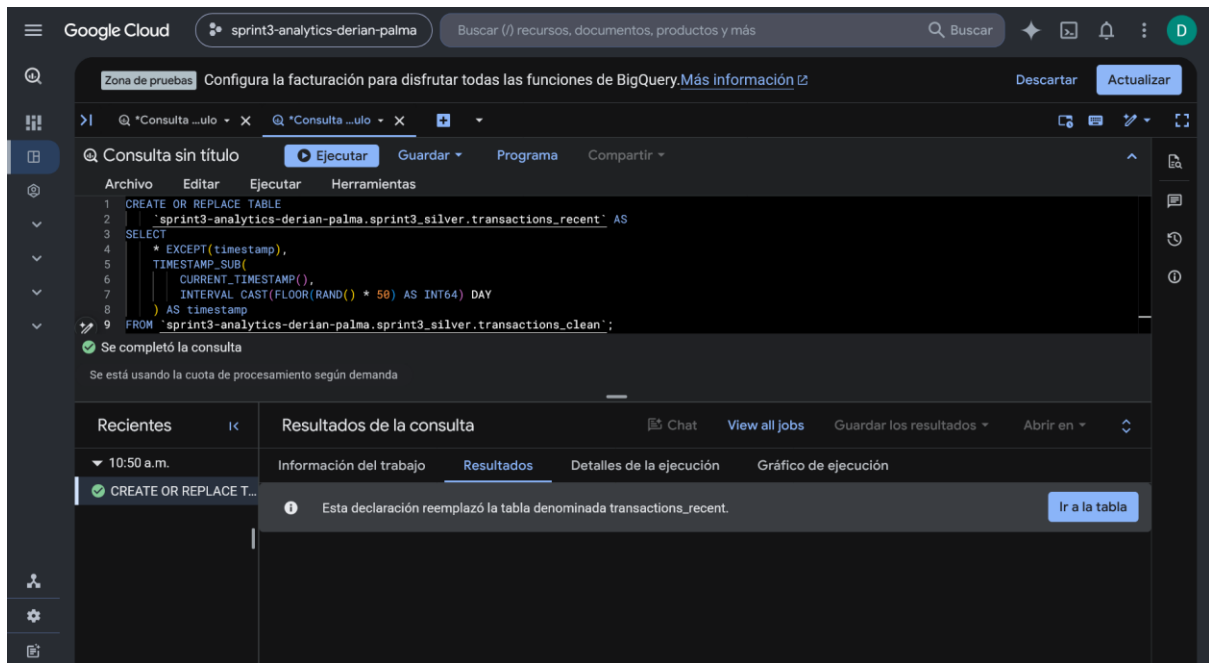
Para adaptar los datos históricos a la caducidad de las particiones del Sandbox, se generará una copia con fechas recientes y, a partir de ella, una tabla optimizada en la capa Gold. La tabla original transactions_clean se conservará sin modificaciones.



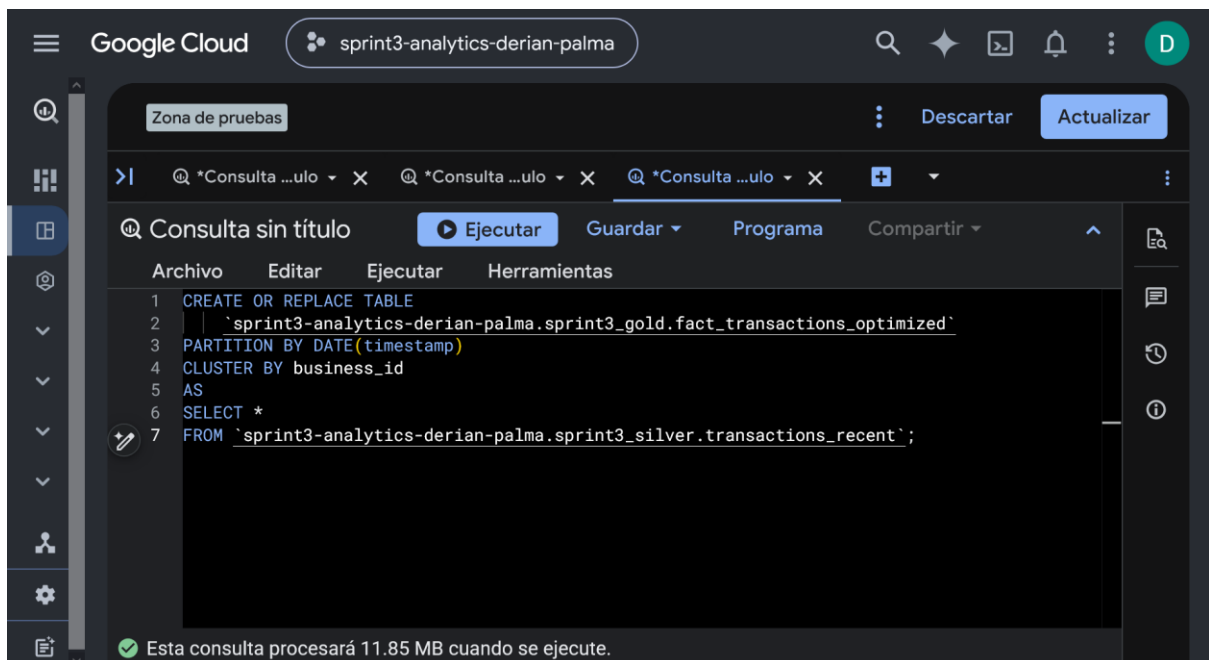
The screenshot shows the Google Cloud BigQuery console interface. At the top, the project name is 'sprint3-analytics-derian-palma'. The main area displays a SQL query in a dark-themed editor. The query is as follows:

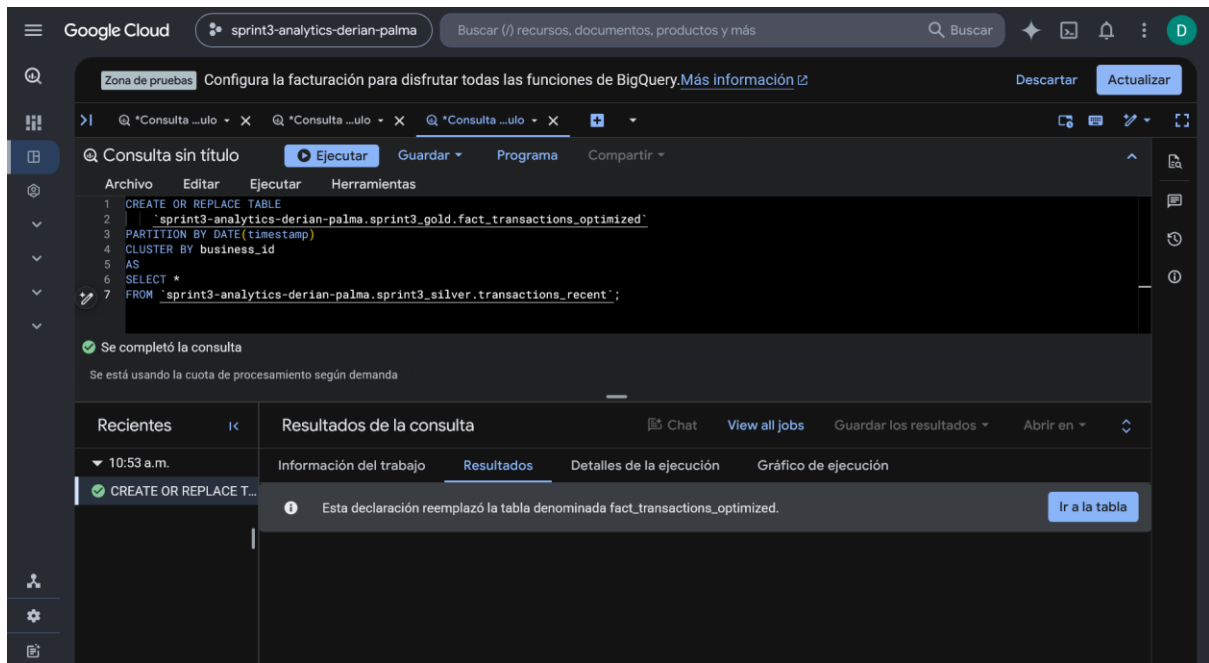
```
1 CREATE OR REPLACE TABLE
2   `sprint3-analytics-derian-palma.sprint3_silver.transactions_recent` AS
3 SELECT
4   * EXCEPT(timestamp),
5   TIMESTAMP_SUB(
6     CURRENT_TIMESTAMP(),
7     INTERVAL CAST(FLOOR(RAND() * 50) AS INT64) DAY
8   ) AS timestamp
9 FROM `sprint3-analytics-derian-palma.sprint3_silver.transactions_clean`;
```

Below the query, a status message indicates: 'Esta consulta procesará 11.09 MB cuando se ejecute.'



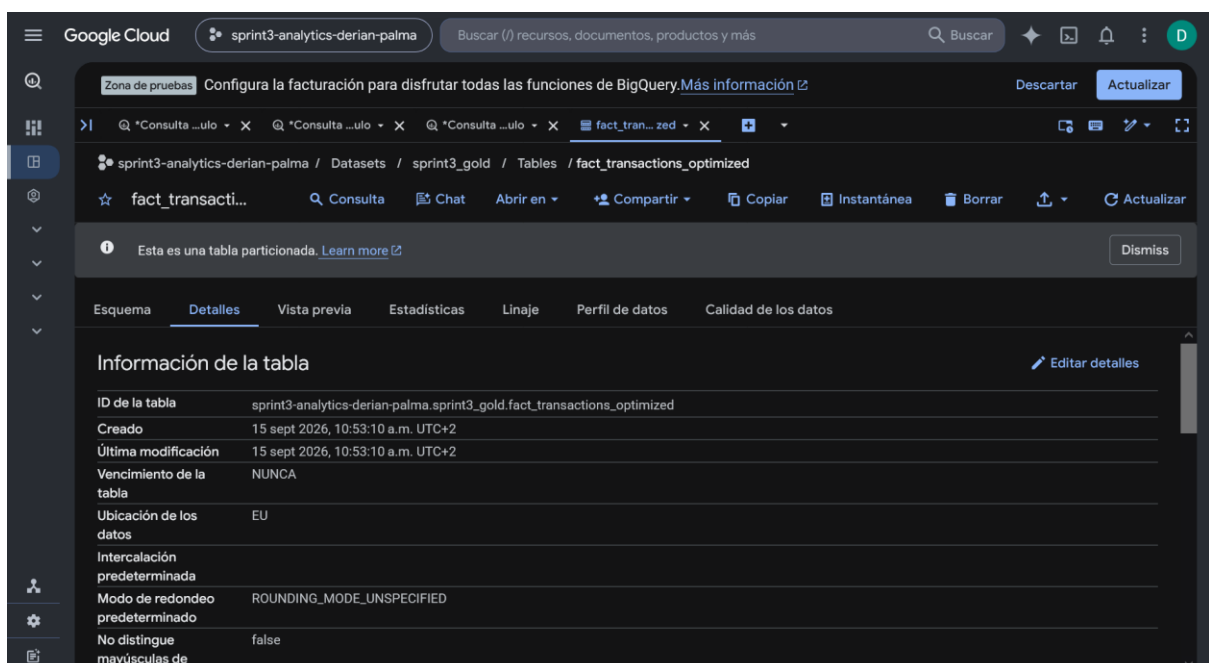
El validador estimó 11,09 MB para crear transactions_recent. La ejecución finalizó correctamente y BigQuery confirmó la creación de la tabla.





El validador estimó 11,85 MB para crear `fact_transactions_optimized`. La ejecución finalizó correctamente y BigQuery confirmó la creación de la tabla.

El primer script crea `transactions_recent`, conservando todas las columnas y sustituyendo `timestamp` por fechas simuladas con un desfase aleatorio de entre 0 y 49 días respecto al instante de ejecución. El segundo script utiliza esa copia para crear `fact_transactions_optimized`, particionada por `DATE(timestamp)` y con clustering por `business_id`. Así se preparan los mismos datos para comparar posteriormente el procesamiento de consultas entre ambas tablas.



The image consists of two screenshots of the Google Cloud BigQuery console, showing the details of the `fact_transactions_optimized` table in the `sprint3_gold` dataset.

Top Screenshot: Table Details

The top screenshot shows the 'Detalles' (Details) tab for the `fact_transactions_optimized` table. The table is partitioned by `DAY` on the `timestamp` field, with a partition expiration of 60 days. It is clustered by `business_id`.

Propiedad	Valor
No distingue mayúsculas de minúsculas	false
Descripción	
Etiquetas	
Claves primarias	
Etiquetas	
Tipo de tabla	Particionada
Particionada por	DAY
Particionada en el campo	timestamp
Vencimiento de la partición	60 días
Filtro de partición	No es obligatorio
Agrupada por	business_id

Bottom Screenshot: Storage Information

The bottom screenshot shows the 'Información de almacenamiento' (Storage Information) tab for the same table. It provides details about the table's size and storage configuration.

Métrica	Valor
Cantidad de filas	100,000
Cantidad de particiones	50
Total de bytes lógicos	11.85 MB
Bytes lógicos activos	11.85 MB
Bytes lógicos a largo plazo	0 B
Bytes físicos actuales	4.41 MB
Total de bytes físicos	4.41 MB
Bytes físicos activos	4.41 MB
Bytes físicos a largo plazo	0 B
Bytes físicos de viajes en el tiempo	0 B

Se verificó en la pestaña Detalles que `fact_transactions_optimized` está particionada diariamente por el campo `timestamp` y agrupada mediante clustering por `business_id`. También se confirmó su ubicación en la región EU y el vencimiento de las particiones a los 60 días. Estas propiedades coinciden con la configuración solicitada en el ejercicio.

Exercici 3: La Prova del Cotó (Benchmark)

Escenari:

Escenari: Ja has creat la taula `sprint3_gold.fact_transactions_optimized` amb les estratègies de Particionament i Clustering. Ara toca demostrar a l'equip la millora de rendiment i l'estalvi de costos que has aconseguit. Simularem una petició típica de negoci: "Volem veure totes les transaccions dels últims 30 dies".

Tasca:

L'objectiu és clar i directe: **aplicar exactament la mateixa consulta a dues taules diferents i comparar-ne el cost computacional.**

Construeix una consulta SQL que seleccioni totes les columnes (`SELECT *`) i filtri les dades dels últims 30 dies.

 **NO executis les consultes encara.** En aquest exercici només utilitzarem el validador de BigQuery ("Dry Run", a dalt a la dreta de l'editor) per visualitzar els **Bytes processats** de manera gratuïta.

Fes la comparativa seguint aquests dos passos:

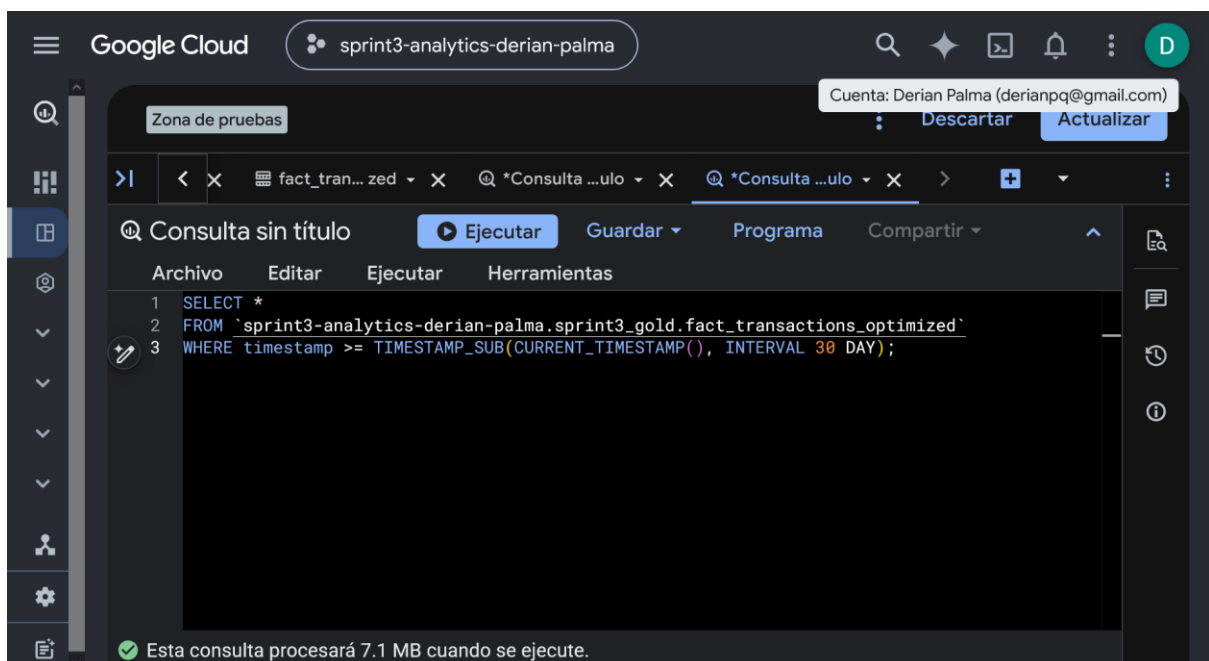
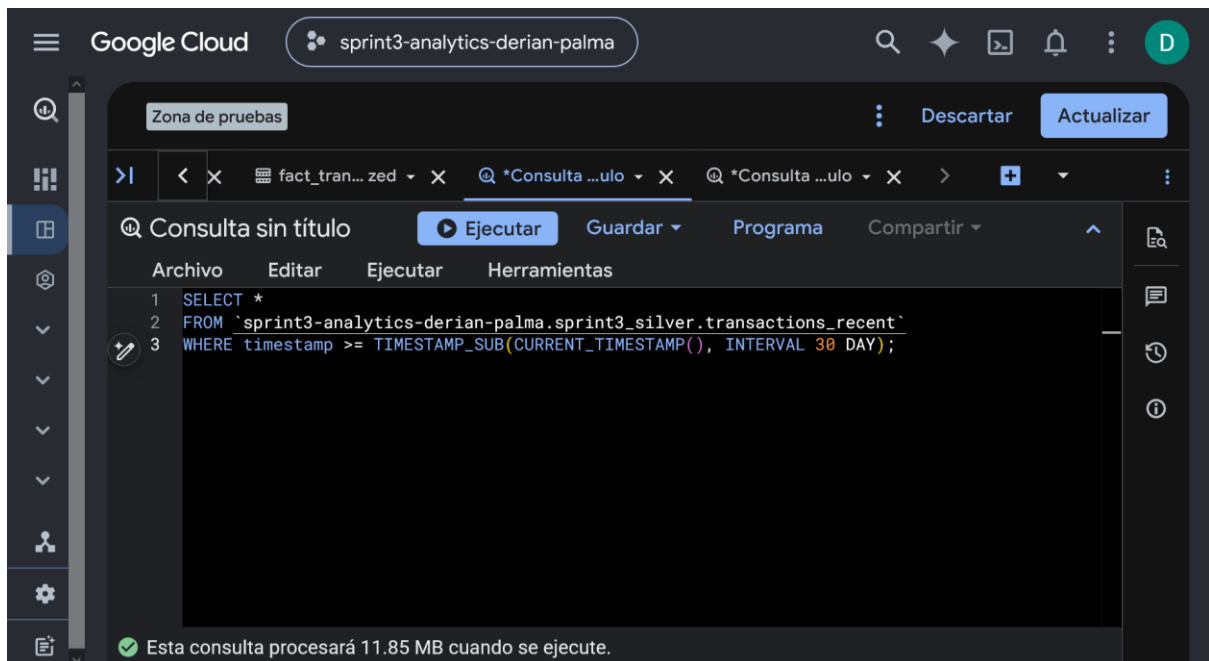
- » **Pas 1 (Taula no optimitzada):** Apunta la teva consulta a la taula `sprint3_silver.transactions_recent`. Observa el validador i fes una captura de pantalla dels bytes que processaria.
- » **Pas 2 (Taula optimitzada):** Sense canviar res de la lògica de la teva consulta, modifica només el `FROM` perquè ara apunti a la taula particionada `sprint3_gold.fact_transactions_optimized`. Observa com canvia el validador i fes la segona captura.

Entrega:

- » El codi SQL base que has dissenyat.
- » Les dues captures de pantalla on es vegi la diferència de "Bytes processats" al validador.
- » Un breu text calculant quin és el % **d'estalvi** que has aconseguit (compara els bytes del Pas 1 respecte als del Pas 2).

Se compara el procesamiento estimado de una consulta que selecciona todas las columnas de las transacciones de los últimos 30 días. Se mantiene la misma lógica y se modifica únicamente la tabla de origen: primero `transactions_recent` y después

fact_transactions_optimized. La comparación se realiza mediante el validador de BigQuery, sin ejecutar las consultas.



Se comparó la misma consulta sobre las transacciones de los últimos 30 días, modificando únicamente la tabla de origen. El validador estimó un procesamiento de 11,85 MB para transactions_recent y de 7,1 MB para fact_transactions_optimized. Esto supone una reducción aproximada de 4,75 MB, equivalente al 40,08 %, calculada a partir de los valores mostrados. La comparación se realizó mediante Dry Run, sin ejecutar las consultas, por lo que estos resultados corresponden a estimaciones de procesamiento y no a tiempos de ejecución ni a importes facturados.

Exercici 4: Smart Caching (Vistes Materialitzades)

Escenari:

El Director General té un quadre de comandament que mostra les "**Vendes Totals per Dia**". Ell (i 50 managers més) refresquen aquest gràfic constantment.

Problema:

Si utilitzes una consulta normal o una Vista estàndard, cada vegada que algú refresca la pàgina, BigQuery recalcula la suma de milions de files. És lent i estàs pagant pel mateix càlcul una vegada i una altra.

Solució:

Utilitzar una **Vista Materialitzada**. Aquesta tecnologia guarda el *resultat* del càlcul a la memòria cau i només l'actualitza si entren dades noves (incremental).

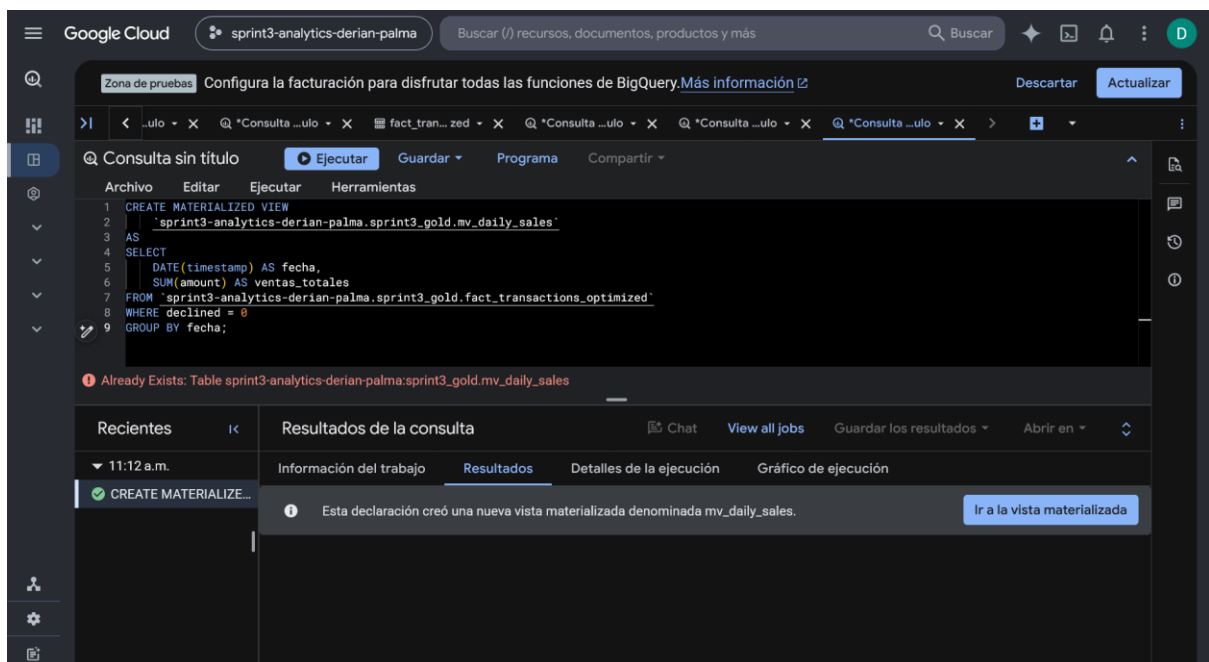
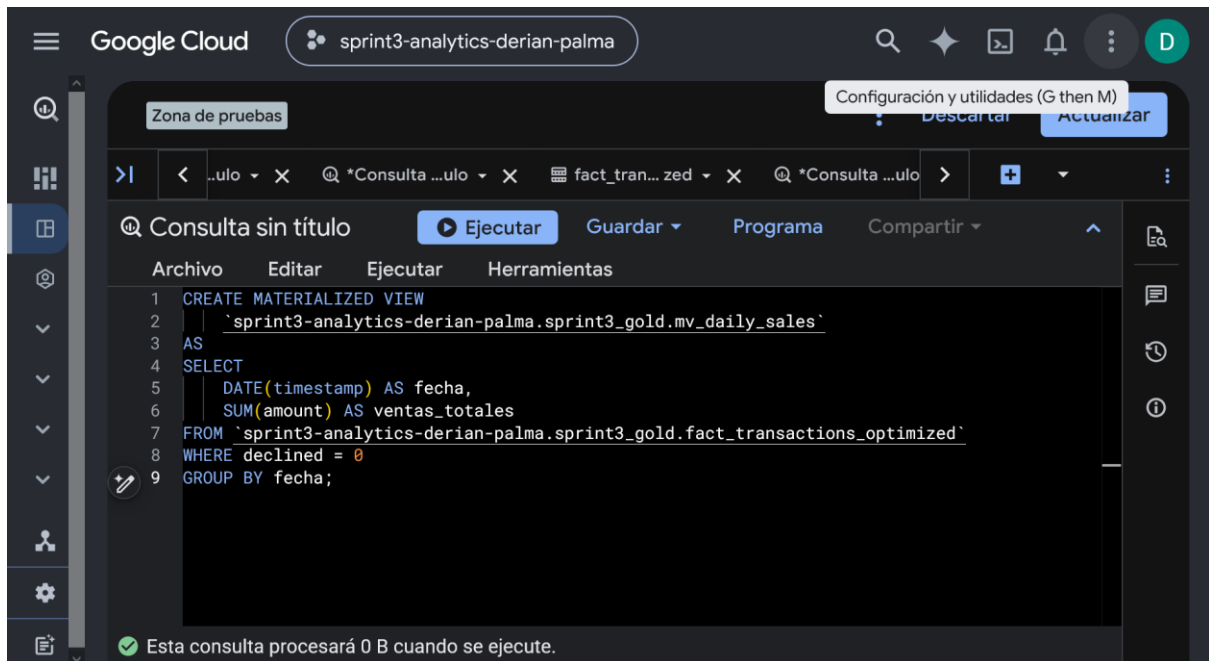
Tasca:

- » Crea una **Vista Materialitzada** anomenada `sprint3_gold.mv_daily_sales` que mostri les **Vendes Totals per Dia**.

Entrega:

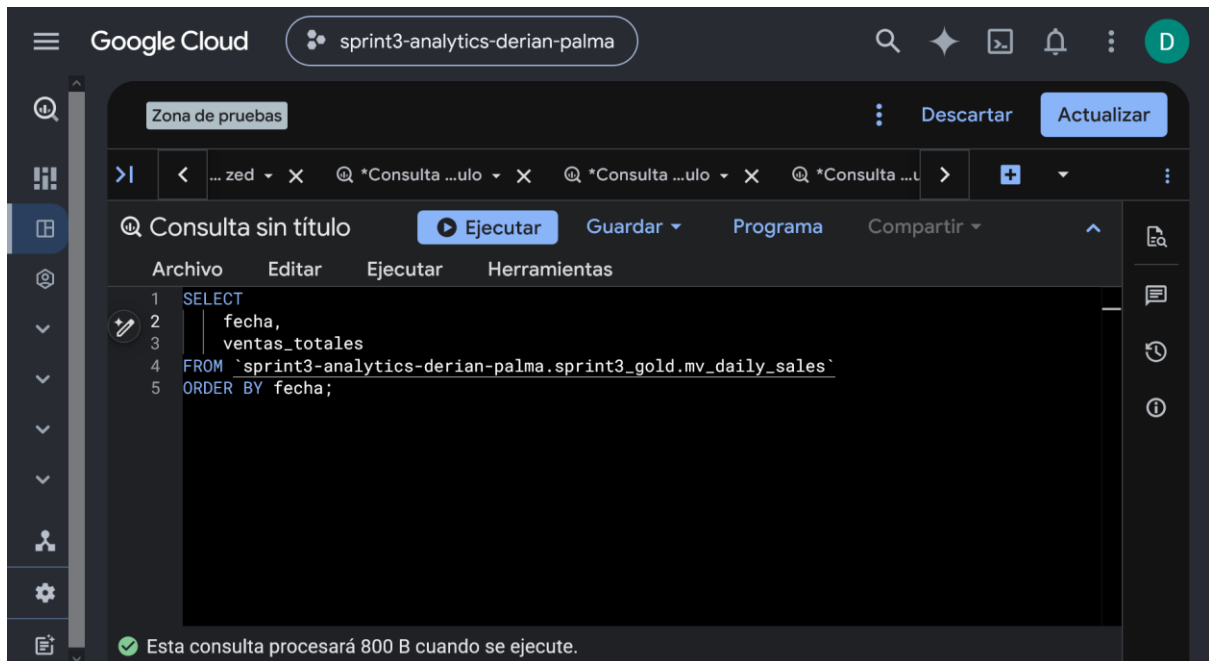
- » Codi per Crear la **Vista Materialitzada**.
- » Fes una consulta que mostri la vista creada i fes una captura de pantalla que mostri en el seu conjunt la vista materialitzada creada, amb la consulta executada i els bits processats.

Se plantea crear la vista materializada `sprint3_gold.mv_daily_sales` sobre `fact_transactions_optimized` para almacenar un resumen de las ventas diarias a partir de la tabla optimizada. Se considerarán únicamente las transacciones aceptadas (`declined = 0`), agrupando por fecha y sumando sus importes. Este resumen se utilizará posteriormente como base para los análisis de evolución diaria y ventas acumuladas.

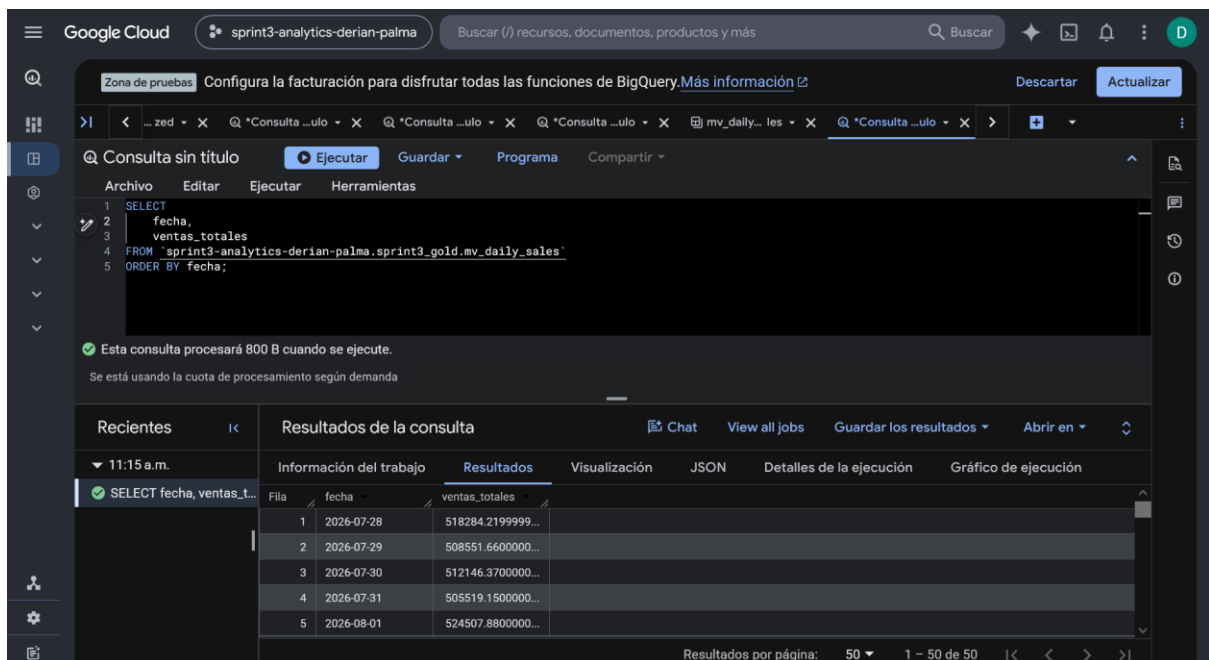


El aviso «Already Exists» aparece en la validación posterior a la creación porque la vista ya existe y el editor conserva la sentencia CREATE MATERIALIZED VIEW. El mensaje de resultados inferior confirma que la creación finalizó correctamente.

Posteriormente, se consultó el resumen de ventas diarias en orden cronológico para verificar los resultados y registrar los bytes procesados.



Tras crear la vista materializada, se validó la consulta del resumen de ventas diarias. El validador estimó un procesamiento de únicamente 800 bytes. Este valor corresponde a la consulta del resumen agregado y se registra como estimación previa a la ejecución.



The screenshot displays the Google Cloud BigQuery console interface. The top navigation bar includes the Google Cloud logo, the project name 'sprint3-analytics-derian-palma', and a search bar. Below the navigation bar, the 'Zona de pruebas' section contains a message about BigQuery features and buttons for 'Descartar' and 'Actualizar'. The main area is divided into two tabs: 'Recientes' and 'Resultados de la consulta'. The 'Resultados de la consulta' tab is active, showing a table of query execution details. The table includes fields such as 'ID de trabajo', 'Usuario', 'Ubicación', 'Hora de creación', 'Hora de inicio', 'Hora de finalización', 'Duración', 'Bytes procesados', 'Bytes facturados', 'Millisegundos de ranura', 'Prioridad del trabajo', 'Usar SQL heredado', 'Tabla de destino', 'Modo de uso de índice', 'Motivos de índice no utilizado', and 'Etiquetas'. The 'Duración' field shows 358 ms, and the 'Bytes procesados' field shows 800 B. The 'Motivos de índice no utilizado' field contains a message: 'There is no index configuration for the base table 'sprint3-analytics-derian-palma:sprint3_gold.fact_transactions_optimized'.'

Información del trabajo	Resultados	Visualización	JSON	Detalles de la ejecución	Gráfico de ejecución
ID de trabajo	sprint3-analytics-derian-palma:EU.job_GQQ98JZOR452HQcWOpu5mF7fqep2 (Ver detalles de rendimiento)				
Usuario	derianpq@gmail.com				
Ubicación	EU				
Hora de creación	15 sept 2026, 11:15:17 a.m. UTC+2				
Hora de inicio	15 sept 2026, 11:15:18 a.m. UTC+2				
Hora de finalización	15 sept 2026, 11:15:18 a.m. UTC+2				
Duración	358 ms				
Bytes procesados	800 B				
Bytes facturados	10 MB				
Millisegundos de ranura	41				
Prioridad del trabajo	INTERACTIVE				
Usar SQL heredado	false				
Tabla de destino	Tabla temporal				
Modo de uso de índice	UNUSED				
Motivos de índice no utilizado	There is no index configuration for the base table 'sprint3-analytics-derian-palma:sprint3_gold.fact_transactions_optimized'.				
Etiquetas					

Se ejecutó la consulta sobre la vista materializada `mv_daily_sales`, mostrando las ventas totales por día en orden cronológico. BigQuery registró 800 bytes procesados, coincidiendo con la estimación del validador, y una duración de 358 ms.

Nivell 2: SQL Analític Avançat

Escenari:

La direcció comença a fer preguntes que no es resolen amb un simple **GROUP BY**. Necessitem anàlisi de tendències i comportament d'usuari.

Exercici 1: Perfilat de Clients VIP (Mètriques Agregades amb CTEs)

Escenari:

L'equip de Màrqueting vol analitzar el comportament dels nostres millors clients per dissenyar l'estratègia del pròxim any. Definim "VIP" com aquells amb una despesa acumulada superior a 500€. Necessiten un informe que, per a cada VIP, mostri el seu nom, contacte i el seu patró de compra: quantes vegades ha comprat, quant gasta de mitjana i quina va ser la seva compra rècord.

Tasca:

» **1. Crea una CTE anomenada VIP_Stats que agrupi per usuari i calculi:**

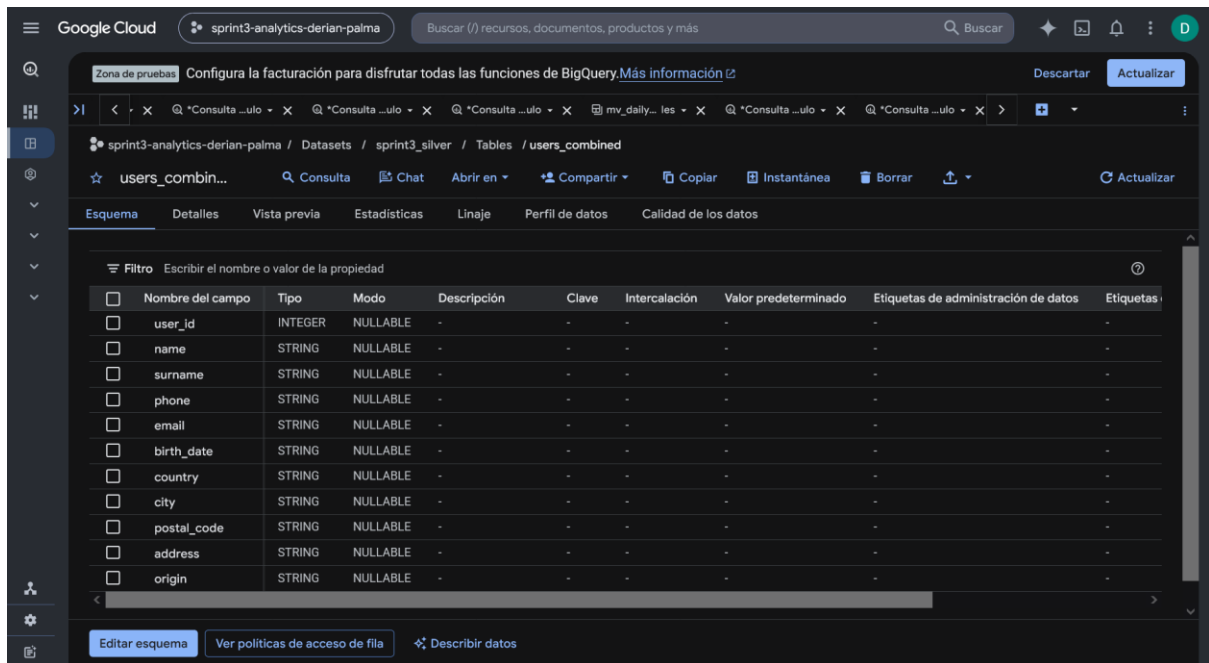
- La Despesa Total (SUM).
- La Quantitat de Transaccions (COUNT).
- El Tiquet Mitjà (AVG), arrodonit a 2 decimals.
- La Compra Màxima (MAX).
- Filtre: Mantén només aquells la Despesa Total dels quals sigui > 500.

» **Creua la CTE amb users_combined per obtenir les dades personals.**

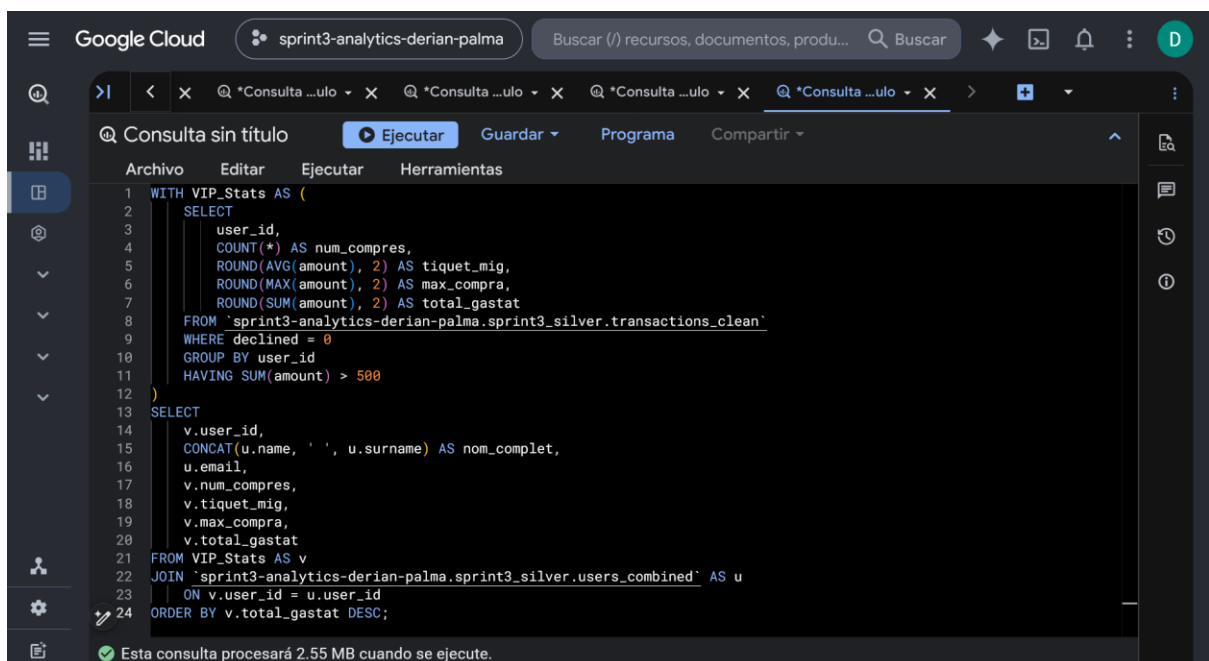
Requisits de Sortida:

- Columnes: user_id, nom_complet, email, num_compres, tiquet_mig, max_compra, total_gastat.
- Ordenat per total_gastat descendent.

Se elaborará un informe de clientes VIP, definidos como aquellos cuyo gasto acumulado en compras aceptadas supera los 500 €. Mediante la CTE VIP_Stats se calcularán las métricas de compra por usuario. Posteriormente, se incorporarán los datos personales de users_combined y se ordenará el resultado por gasto total descendente.



Se revisó el esquema de `users_combined` y se confirmó que `user_id` es de tipo `INTEGER`, compatible con el identificador de usuario de `transactions_clean`. También se identificaron los campos `name`, `surname` y `email`, necesarios para incorporar el nombre completo y el correo al informe de clientes VIP.



La CTE `VIP_Stats` agrupa las transacciones aceptadas por usuario y calcula el número de compras, el ticket medio redondeado a dos decimales (también se redondean otras cifras siguiendo las buenas prácticas), la compra máxima y el gasto total. Mediante `HAVING SUM(amount) > 500` se seleccionan los clientes VIP. A continuación, se une el resultado con `users_combined` para incorporar el nombre completo y el correo electrónico, ordenando el informe por gasto total descendente.

Google Cloud sprint3-analytics-derian-palma Buscar (/) recursos, documentos, productos y más

Cuenta: Derian Palma (derianpq@gmail.com) Descartar Actualizar

Zona de pruebas Configura la facturación para disfrutar todas las funciones de BigQuery. Más información

Resultados de la consulta Chat View all jobs Guardar los resultados Abrir en

Información del trabajo Resultados Visualización JSON Detalles de la ejecución Gráfico de ejecución

Fila	user_id	nom_complet	email	num_compres	tiquet_mig	max_compra	total_gastat
1	289	Dxwgi Hwcru	dxwgi.hwcru@example.com	91	446.91	554.71	40669.16
2	318	Bnyr Astuw	bnyr.astuw@example.com	86	449.0	615.51	38614.39
3	455	Fwfil Phylgmnhd	fwfil.phylgmnhd@example.com	66	522.6	724.44	34491.27
4	417	Fqatil Rdwcjvoa	fqatil.rdwcjvoa@example.com	73	421.87	675.78	30796.3
5	185	Molly Gilliam	donec@outlook.couk	107	274.72	659.31	29394.79
6	285	Brcxprnj Lnpna	brcxprnj.lnpna@example.com	70	417.85	630.52	29249.51
7	193	Minerva Wilkins	aliquam.adipiscing@yahoo.edu	67	380.37	646.0	25485.05
8	454	Sfzzoh Xgvfridxs	sfzzoh.xgvfridxs@example.com	78	304.0	604.9	23711.9
9	349	Yxxcjk Fnceqbkxmt	yxxcjk.fnceqbkxmt@example.c...	53	443.48	675.05	23504.47
10	317	Qikcvc Nvfctfcdh	qikcvc.nvfctfcdh@example.com	54	351.69	600.77	18991.45
11	388	Rbjupnac Cfjyidx	rbjupnac.cfjyidx@example.com	62	297.3	479.03	18432.84
12	319	Fxtg Eedgl	fxtg.eedgl@example.com	62	286.62	576.22	17770.39

Resultados por página: 50 1 - 50 de 5000

La consulta se ejecutó correctamente y devolvió 5.000 filas con los datos personales y las métricas de compra de los clientes que cumplen el criterio VIP. El validador estimó un procesamiento de 2,55 MB.

El informe muestra el identificador, nombre completo y correo de cada cliente VIP, junto con el número de compras, ticket medio, compra máxima y gasto total. Se comprobó el orden descendente por gasto total. El primer registro corresponde al usuario 289, con 91 compras y un gasto acumulado aproximado de 40.669,16 €.

Exercici 2: Anàlisi de Tendències (Window Functions sobre Vistes)

Escenari:

La Direcció Financera vol monitoritzar la "velocitat de vendes" diària. Necessiten un informe que compari el rendiment de cada dia contra el dia anterior per detectar caigudes brusques o pics de creixement (Day-over-Day Growth).

☑ Estratègia d'Optimització:

Per a aquest càlcul NO has d'anar a la taula de fets original ni recalculer sumes. Com a bon Data Engineer, has d'utilitzar la Vista Materialitzada `sprint3_gold.mv_daily_sales` que vas crear a l'exercici anterior. Això garanteix que l'anàlisi de tendències sigui instantània i no consumeixi recursos de còmput innecessaris.

⚙ Tasca:

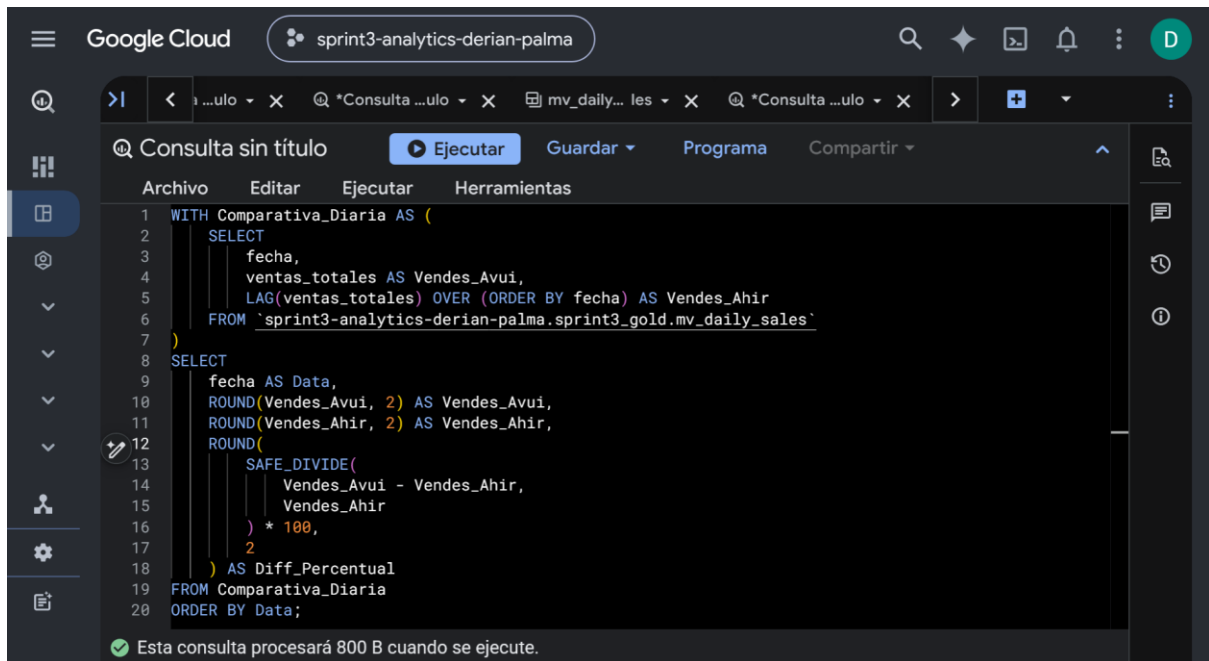
Crea una consulta sobre la vista materialitzada que generi un informe amb les següents 4 columnes:

- » 1.**Data**: La data de venda.
- » 2.**Vendes_Avui**: El total venut aquell dia.
- » 3.**Vendes_Ahir**: El total venut el dia anterior (usant funcions de finestra).
- » 4.**Diff_Percentual**: El percentatge de creixement o decreixement respecte al dia anterior, arrodonit a 2 decimals.

💡 Pista Tècnica:

Utilitza la funció `LAG(camp) OVER (ORDER BY data)` per llegir el valor de la fila anterior sense necessitat de fer `JOINS` complexos.

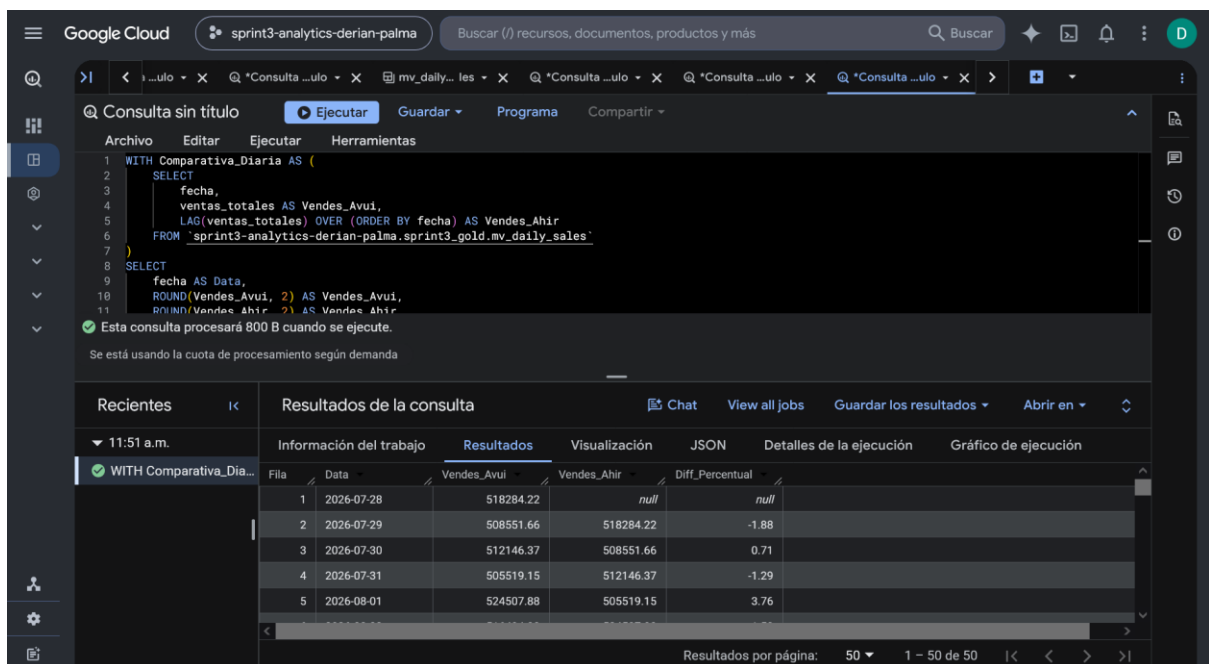
Se analizará la variación diaria de ventas utilizando la vista materializada `mv_daily_sales`, sin recalcular las agregaciones de la tabla de transacciones. Mediante `LAG` se recuperará el total de la fecha anterior disponible y se calculará la variación porcentual, redondeada a dos decimales.



The screenshot shows the Google Cloud BigQuery console interface. At the top, the project name is 'sprint3-analytics-derian-palma'. The editor displays a SQL query titled 'Consulta sin título'. The query uses a CTE named 'Comparativa_Diaria' to calculate the daily percentage change in sales. It selects 'fecha', 'ventas_totales' (aliased as 'Vendes_Avui'), and 'LAG(ventas_totales) OVER (ORDER BY fecha)' (aliased as 'Vendes_Ahir') from the table 'sprint3-analytics-derian-palma.sprint3_gold.mv_daily_sales'. The main query then calculates the percentage difference using 'SAFE_DIVIDE(Vendes_Avui - Vendes_Ahir, Vendes_Ahir) * 100' and rounds the results to 2 decimal places.

```
1 WITH Comparativa_Diaria AS (  
2   SELECT  
3     fecha,  
4     ventas_totales AS Vendes_Avui,  
5     LAG(ventas_totales) OVER (ORDER BY fecha) AS Vendes_Ahir  
6   FROM `sprint3-analytics-derian-palma.sprint3_gold.mv_daily_sales`  
7 )  
8 SELECT  
9   fecha AS Data,  
10  ROUND(Vendes_Avui, 2) AS Vendes_Avui,  
11  ROUND(Vendes_Ahir, 2) AS Vendes_Ahir,  
12  ROUND(  
13    SAFE_DIVIDE(  
14      Vendes_Avui - Vendes_Ahir,  
15      Vendes_Ahir  
16    ) * 100,  
17    2  
18  ) AS Diff_Percentual  
19 FROM Comparativa_Diaria  
20 ORDER BY Data;
```

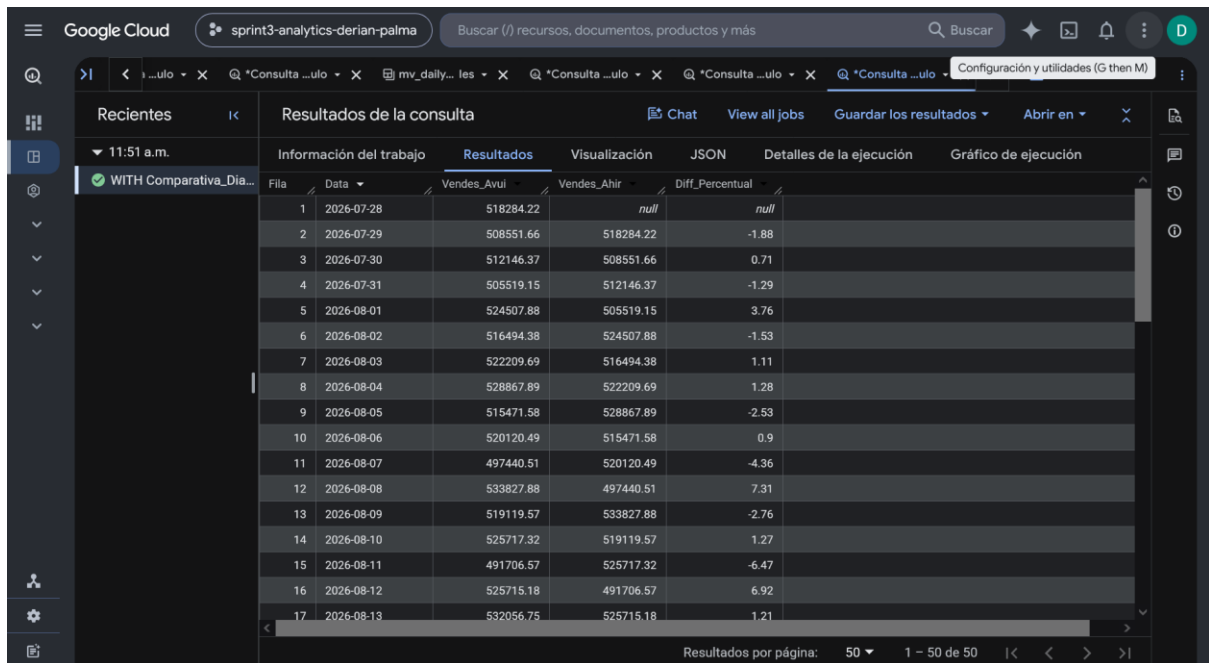
Esta consulta procesará 800 B cuando se ejecute.



The screenshot shows the Google Cloud BigQuery console interface with the query results displayed. The results are shown in a table with 5 columns: 'Fila', 'Data', 'Vendes_Avui', 'Vendes_Ahir', and 'Diff_Percentual'. The table contains 5 rows of data. The first row shows a null value for 'Diff_Percentual' because there is no previous date in the window. The subsequent rows show the percentage change in sales.

Fila	Data	Vendes_Avui	Vendes_Ahir	Diff_Percentual
1	2026-07-28	518284.22	null	null
2	2026-07-29	508551.66	518284.22	-1.88
3	2026-07-30	512146.37	508551.66	0.71
4	2026-07-31	505519.15	512146.37	-1.29
5	2026-08-01	524507.88	505519.15	3.76

Resultados por página: 50 1 - 50 de 50



The screenshot shows the Google Cloud BigQuery console interface. The top navigation bar includes the Google Cloud logo, the project name 'sprint3-analytics-derian-palma', and a search bar. The main content area displays the 'Resultados de la consulta' (Query Results) tab. The query is named 'WITH Comparativa_Dia...'. The results are shown in a table with the following columns: 'Fila' (Row), 'Data' (Date), 'Vendes_Avui' (Current Sales), 'Vendes_Ahir' (Previous Sales), and 'Diff_Percentual' (Percentage Difference). The table contains 17 rows of data, starting from 2026-07-28 and ending on 2026-08-13. The first row shows NULL values for 'Vendes_Ahir' and 'Diff_Percentual' because there is no previous day's data. Subsequent rows show sales figures and their percentage change from the previous day. For example, on 2026-07-29, sales decreased by 1.88% from 518,284.22 to 508,551.66. On 2026-07-30, sales increased by 0.71% to 512,146.37. The table is paginated, showing 50 results per page, with the first page displaying 17 rows.

Fila	Data	Vendes_Avui	Vendes_Ahir	Diff_Percentual
1	2026-07-28	518284.22	null	null
2	2026-07-29	508551.66	518284.22	-1.88
3	2026-07-30	512146.37	508551.66	0.71
4	2026-07-31	505519.15	512146.37	-1.29
5	2026-08-01	524507.88	505519.15	3.76
6	2026-08-02	516494.38	524507.88	-1.53
7	2026-08-03	522209.69	516494.38	1.11
8	2026-08-04	528867.89	522209.69	1.28
9	2026-08-05	515471.58	528867.89	-2.53
10	2026-08-06	520120.49	515471.58	0.9
11	2026-08-07	497440.51	520120.49	-4.36
12	2026-08-08	533827.88	497440.51	7.31
13	2026-08-09	519119.57	533827.88	-2.76
14	2026-08-10	525717.32	519119.57	1.27
15	2026-08-11	491706.57	525717.32	-6.47
16	2026-08-12	525715.18	491706.57	6.92
17	2026-08-13	532056.75	525715.18	1.21

La consulta generó un informe de 50 días con las ventas diarias, las ventas del día anterior y su variación porcentual redondeada a dos decimales. La primera fecha, el 28 de julio de 2026, presenta valores NULL en la comparación por carecer de un registro anterior. Como ejemplo, el 29 de julio las ventas disminuyeron un 1,88 %, pasando de 518.284,22 a 508.551,66, mientras que el 30 de julio aumentaron un 0,71 %, hasta 512.146,37. La consulta utiliza la vista materializada de ventas diarias. El validador estimó un procesamiento de 800 bytes al consultar la vista materializada de ventas diarias, sin recalcular las sumas de las transacciones originales.

Exercici 3: Totals Acumulats (Running Totals sobre Vistes)

Escenari:

El Director Financer (CFO) necessita visualitzar la corba de creixement anual. No li interessa tant el que es va vendre avui, sinó quant portem facturat en total des de l'inici de l'any fins avui (YTD - Year To Date).

Tasca:

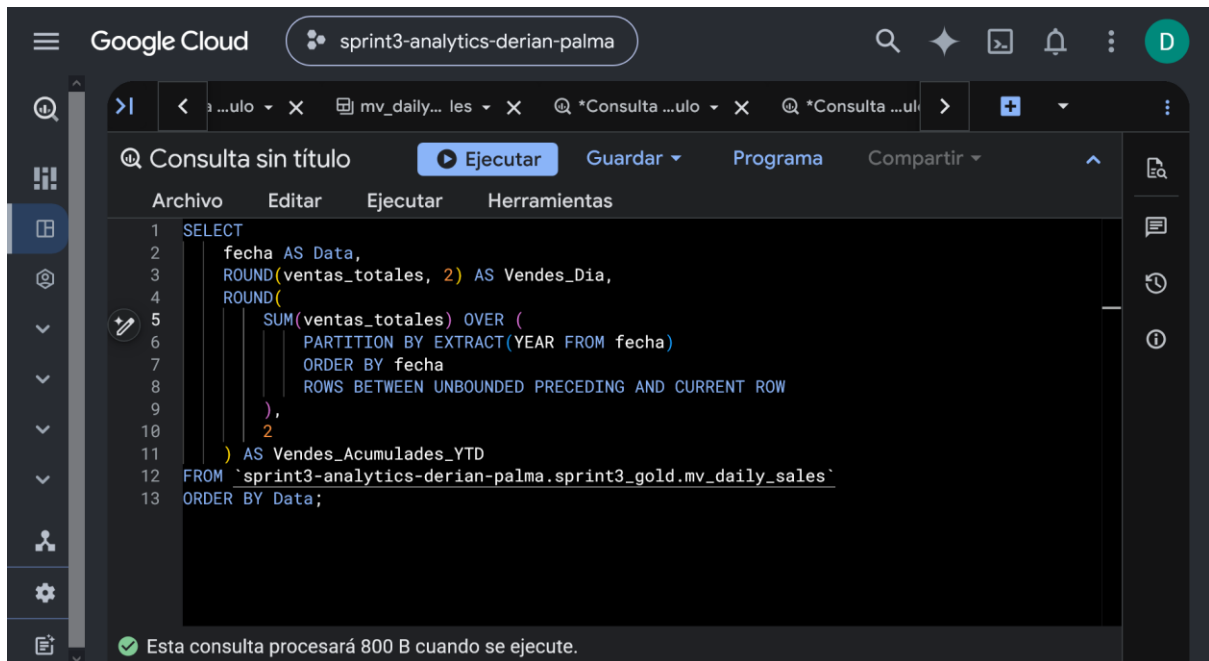
Utilitzant la vista materialitzada `mv_daily_sales` (per no recalculer agregacions base), genera un informe amb tres columnes:

- » 1.**Data**.
- » 2.**Vendes del Dia**: Arrodonit a 2 decimals.
- » 3.**Vendes Acumulades YTD**: Una suma progressiva de les vendes que ha de reiniciar-se cada **1 de Gener**. El resultat final ha d'estar arrodonit a 2 decimals.

Pista Tècnica:

Per aconseguir el reinici anual, necessites dividir la finestra usant `PARTITION BY EXTRACT(YEAR FROM data)`. L'estructura completa seria: `SUM(...) OVER (PARTITION BY ... ORDER BY ... ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)`.

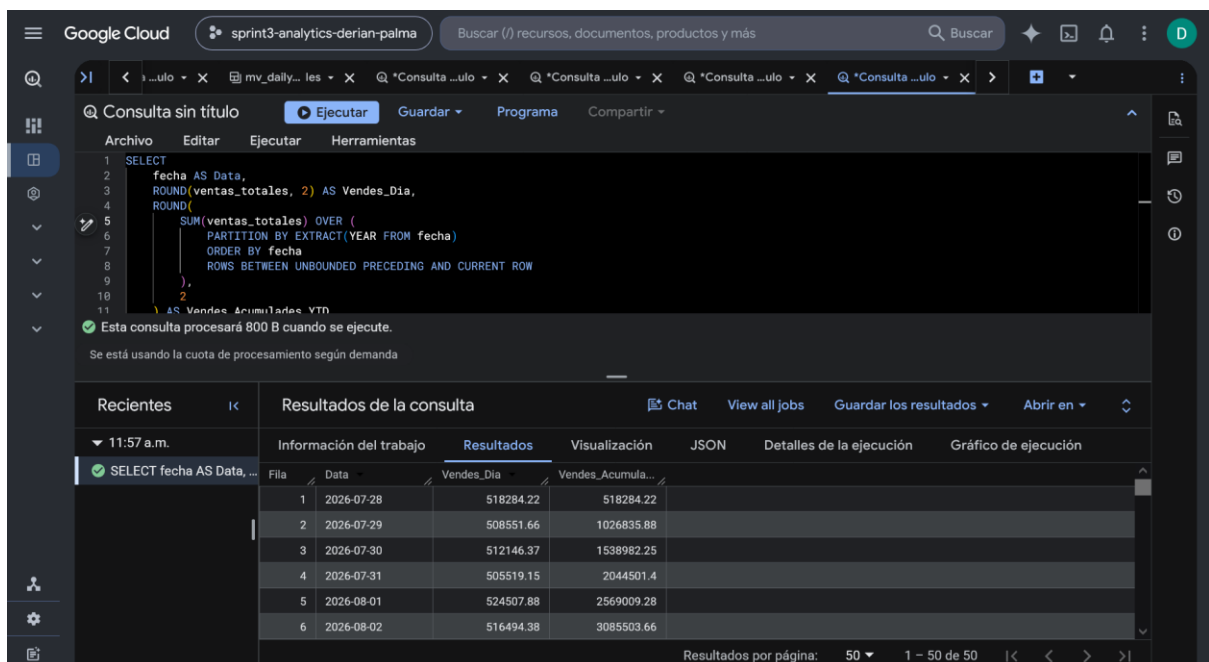
Se calcularán las ventas acumuladas anuales a partir de la vista materializada `mv_daily_sales`. Se utilizará una suma de ventana ordenada por fecha y particionada por año, acumulando desde el primer registro disponible de cada año hasta la fecha de cada fila. Tanto las ventas diarias como el acumulado se mostrarán redondeados a dos decimales.



The screenshot shows the Google Cloud BigQuery console interface. The top navigation bar includes the Google Cloud logo, the project name 'sprint3-analytics-derian-palma', and various utility icons. The main editor area displays a SQL query titled 'Consulta sin título'. The query is as follows:

```
1 SELECT
2   fecha AS Data,
3   ROUND(ventas_totales, 2) AS Vendes_Dia,
4   ROUND(
5     SUM(ventas_totales) OVER (
6       PARTITION BY EXTRACT(YEAR FROM fecha)
7       ORDER BY fecha
8       ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
9     ),
10    2
11  ) AS Vendes_Acumulades_YTD
12 FROM `sprint3-analytics-derian-palma.sprint3_gold.mv_daily_sales`
13 ORDER BY Data;
```

Below the editor, a status message indicates: 'Esta consulta procesará 800 B cuando se ejecute.'



The screenshot shows the Google Cloud BigQuery console interface with the query results displayed. The top navigation bar is the same as the previous screenshot. The main editor area shows the same SQL query. Below the editor, a status message indicates: 'Esta consulta procesará 800 B cuando se ejecute.' and 'Se está usando la cuota de procesamiento según demanda'.

The results are displayed in a table with the following columns: Fila, Data, Vendes_Dia, and Vendes_Acumula... The table contains 6 rows of data.

Fila	Data	Vendes_Dia	Vendes_Acumula...
1	2026-07-28	518284.22	518284.22
2	2026-07-29	508551.66	1026835.88
3	2026-07-30	512146.37	1538982.25
4	2026-07-31	505519.15	2044501.4
5	2026-08-01	524507.88	2569009.28
6	2026-08-02	516494.38	3085503.66

Results per page: 50. 1 - 50 de 50.

Fila	Data	Vendas_Dia	Vendas_Acumula...
1	2026-07-28	518284.22	518284.22
2	2026-07-29	508551.66	1026835.88
3	2026-07-30	512146.37	1538982.25
4	2026-07-31	505519.15	2044501.4
5	2026-08-01	524507.88	2569009.28
6	2026-08-02	516494.38	3085503.66
7	2026-08-03	522209.69	3607713.35
8	2026-08-04	528867.89	4136581.24
9	2026-08-05	515471.58	4652052.82
10	2026-08-06	520120.49	5172173.31
11	2026-08-07	497440.51	5669613.82
12	2026-08-08	533827.88	6203441.7
13	2026-08-09	519119.57	6722561.27
14	2026-08-10	525717.32	7248278.59
15	2026-08-11	491706.57	7739985.16
16	2026-08-12	525715.18	8265700.34
17	2026-08-13	532056.75	8797757.09

La consulta devolvió 50 registros con las ventas diarias y su acumulado anual, redondeados a dos decimales. En la primera fecha, el 28 de julio de 2026, el acumulado coincide con las ventas del día: 518.284,22 €. En las siguientes incorpora progresivamente los importes anteriores; por ejemplo, el 30 de julio alcanza 1.538.982,25 €. El cálculo comprende los datos disponibles desde el 28 de julio y contempla el reinicio por año, aunque el periodo actual no incluye un cambio de año que permita observarlo.

Exercici 4: Fidelització i Valor del Client (Filtratge Avançat)

Escenari:

L'equip de Màrqueting llança la campanya "A la tercera va la vençuda". Volen enviar un regal als usuaris que hagin completat la seva tercera transacció. No obstant això, el pressupost del regal no és fix: depèn de la "qualitat" del client en els seus inicis. Necessiten saber el Tiquet Mitjà d'aquestes 3 primeres compres per segmentar si envien un detall bàsic o un regal premium.

Tasca:

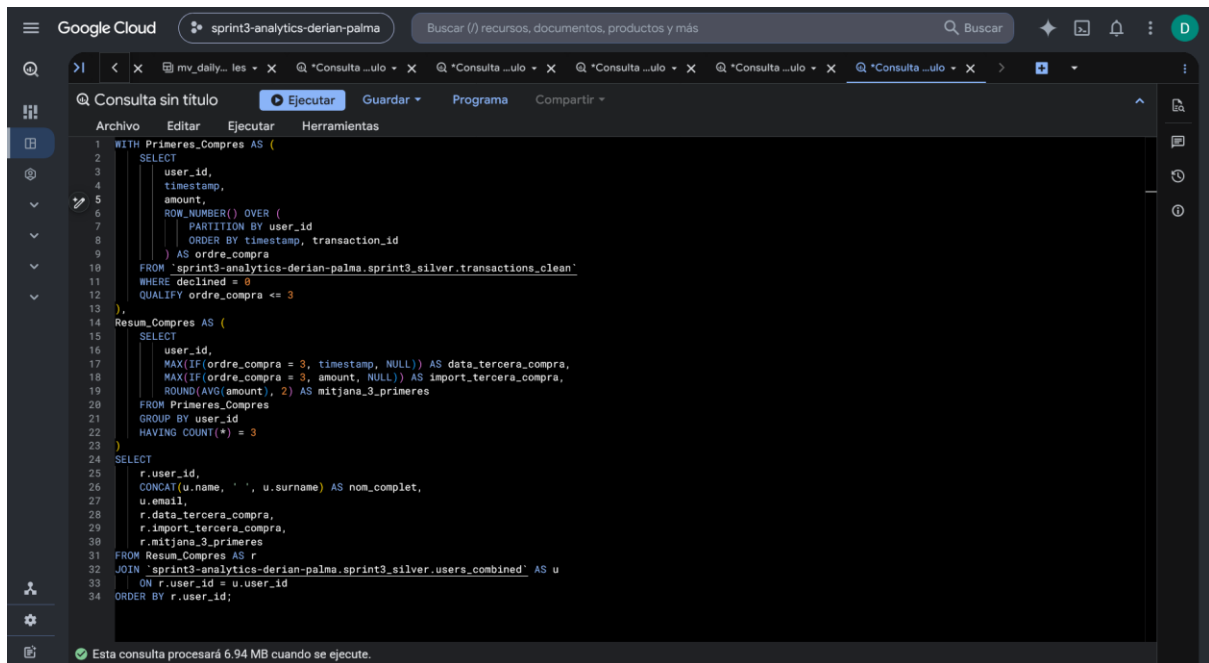
Genera un llistat dels usuaris que han arribat (o superat) la seva tercera compra, mostrant:

- » 1. **Dades d'usuari** (`user_id`, `nom_complet`, `email`).
- » 2. **La data i l'import exacte de la 3a compra**.
- » 3. **Mitjana de les 3 primeres**: La mitjana de despesa de les seves transaccions 1, 2 i 3.

Pista Tècnica:

- » **Has d'utilitzar la funció `ROW_NUMBER()` per establir l'ordre cronològic de les compres**.
- » **Has d'utilitzar la clàusula `QUALIFY` per filtrar**.
- » **Repte d'Optimització**: Busca l'estratègia de menor cost computacional. Tingues en compte que calcular mitjanes sobre tot l'historial històric és costós. Intenta filtrar les files necessàries abans de realitzar l'agregació final.

Se identificarán los usuarios con al menos tres compras aceptadas y se analizarán sus tres primeras transacciones. Se utilizará `transactions_clean` para conservar las fechas originales y el orden histórico de compra.



```
1 WITH Primeres_Compras AS (  
2     SELECT  
3         user_id,  
4         timestamp,  
5         amount,  
6         ROW_NUMBER() OVER (  
7             PARTITION BY user_id  
8             ORDER BY timestamp, transaction_id  
9         ) AS ordre_compra  
10    FROM `sprint3-analytics-derian-palma.sprint3_silver.transactions_clean`  
11   WHERE declined = 0  
12   QUALIFY ordre_compra <= 3  
13 ),  
14 Resum_Compras AS (  
15     SELECT  
16         user_id,  
17         MAX(IF(ordre_compra = 3, timestamp, NULL)) AS data_tercera_compra,  
18         MAX(IF(ordre_compra = 3, amount, NULL)) AS import_tercera_compra,  
19         ROUND(AVG(amount), 2) AS mitjana_3_primeres  
20    FROM Primeres_Compras  
21   GROUP BY user_id  
22   HAVING COUNT(*) = 3  
23 )  
24 SELECT  
25     r.user_id,  
26     CONCAT(u.name, ' ', u.surname) AS nom_complet,  
27     u.email,  
28     r.data_tercera_compra,  
29     r.import_tercera_compra,  
30     r.mitjana_3_primeres  
31  FROM Resum_Compras AS r  
32  JOIN `sprint3-analytics-derian-palma.sprint3_silver.users_combined` AS u  
33     ON r.user_id = u.user_id  
34  ORDER BY r.user_id;
```

Esta consulta procesará 6.94 MB cuando se ejecute.

El validador estimó un procesamiento de 6,94 MB para esta consulta. El código completo es:

```

WITH Primeres_Compres AS (
  SELECT
    user_id,
    timestamp,
    amount,
    ROW_NUMBER() OVER (
      PARTITION BY user_id
      ORDER BY timestamp, transaction_id
    ) AS ordre_compra
  FROM `sprint3-analytics-derian-palma.sprint3_silver.transactions_clean`
  WHERE declined = 0
  QUALIFY ordre_compra <= 3
),
Resum_Compres AS (
  SELECT
    user_id,
    MAX(IF(ordre_compra = 3, timestamp, NULL)) AS data_tercera_compra,
    MAX(IF(ordre_compra = 3, amount, NULL)) AS import_tercera_compra,
    ROUND(AVG(amount), 2) AS mitjana_3_primeres
  FROM Primeres_Compres
  GROUP BY user_id
  HAVING COUNT(*) = 3
)
SELECT
  r.user_id,
  CONCAT(u.name, ' ', u.surname) AS nom_complet,
  u.email,
  r.data_tercera_compra,
  r.import_tercera_compra,
  r.mitjana_3_primeres
FROM Resum_Compres AS r
JOIN `sprint3-analytics-derian-palma.sprint3_silver.users_combined` AS u
  ON r.user_id = u.user_id
ORDER BY r.user_id;

```

La primera CTE numera las compras por usuario mediante ROW_NUMBER y utiliza QUALIFY para conservar únicamente las tres primeras. La segunda calcula la media sobre esas filas, extrae la fecha y el importe de la tercera compra y excluye a los usuarios con menos de tres compras. Finalmente, se incorporan el nombre completo y el correo electrónico. El filtrado previo reduce las filas que llegan a la agregación final.

The screenshot shows the Google Cloud BigQuery console interface. At the top, the project is 'sprint3-analytics-derian-palma'. A search bar is present. Below the navigation bar, the query editor shows a SQL query titled 'Consulta sin título'. The query is as follows:

```
1 WITH Primeres_Compras AS (
2   SELECT
3     user_id,
4     timestamp,
5     amount,
6     ROW_NUMBER() OVER (
7       PARTITION BY user_id
8       ORDER BY timestamp, transaction_id
9     ) AS ordre_compra
10  FROM `sprint3-analytics-derian-palma.sprint3_silver.transactions_clean`
11  WHERE declined = 0
```

Below the query editor, a status message indicates: 'Esta consulta procesará 6.94 MB cuando se ejecute. Se está usando la cuota de procesamiento según demanda'.

The results section shows the query execution details. The 'Resultados de la consulta' tab is active, displaying a table with the following columns: 'Fila', 'user_id', 'nom_complet', 'email', 'data_tercera_compra', and 'import_tercera_c...'. The table contains 5 rows of data:

Fila	user_id	nom_complet	email	data_tercera_compra	import_tercera_c...
1	1	Zeus Gamble	interdum.enim@protonmail.edu	2015-12-20 18:06:59 UTC	161.33
2	2	Garrett Mcconnell	integer.vitae.nibh@protonmail.o...	2015-09-21 01:23:22 UTC	267.6
3	3	Ciaran Harrison	interdum.feugiat@aol.org	2016-01-22 17:27:05 UTC	59.76
4	4	Howard Stafford	omare.egestas@icloud.edu	2016-09-28 16:35:32 UTC	171.13
5	5	Hayfa Pierce	et.malesuada.fames@hotmail....	2015-08-17 21:34:14 UTC	51.94

The bottom of the results section shows 'Resultados por página: 50' and '1 - 50 de 5000'.

This screenshot shows the same Google Cloud BigQuery console, but with the 'Resultados de la consulta' tab expanded to show more rows. The table has 17 rows of data, including the first 5 rows shown in the previous screenshot. The columns are: 'Fila', 'user_id', 'nom_complet', 'email', 'data_tercera_compra', 'import_tercera_c...', and 'mitjana_3_primer...'. The data is as follows:

Fila	user_id	nom_complet	email	data_tercera_compra	import_tercera_c...	mitjana_3_primer...
1	1	Zeus Gamble	interdum.enim@protonmail.edu	2015-12-20 18:06:59 UTC	161.33	239.13
2	2	Garrett Mcconnell	integer.vitae.nibh@protonmail.o...	2015-09-21 01:23:22 UTC	267.6	287.46
3	3	Ciaran Harrison	interdum.feugiat@aol.org	2016-01-22 17:27:05 UTC	59.76	166.44
4	4	Howard Stafford	omare.egestas@icloud.edu	2016-09-28 16:35:32 UTC	171.13	225.76
5	5	Hayfa Pierce	et.malesuada.fames@hotmail....	2015-08-17 21:34:14 UTC	51.94	184.49
6	6	Joel Tyson	gravidia.nunc.sed@yahoo.ca	2015-05-22 13:15:32 UTC	386.09	316.65
7	7	Rafael Jimenez	eget@outlook.ca	2015-07-18 13:34:14 UTC	251.42	282.43
8	8	Nissim Franks	egestas.aliquam.fringilla@goog...	2016-04-16 09:27:36 UTC	410.88	337.88
9	9	Mannix McClain	aliquam.nisl@outlook.com	2015-07-21 08:28:10 UTC	139.59	90.94
10	10	Robert Mccarthy	fermentum@protonmail.com	2015-11-08 14:07:24 UTC	470.05	365.8
11	11	Joan Baird	et@outlook.net	2016-03-24 06:24:54 UTC	211.3	187.42
12	12	Benedict Wheeler	tincidunt.donec.vitae@hotmail....	2016-02-11 22:11:17 UTC	344.47	306.74
13	13	Allegra Stanton	proin.eget@protonmail.ca	2015-04-20 01:10:56 UTC	136.69	78.36
14	14	Sara Flynn	integer@outlook.net	2016-06-04 01:07:16 UTC	304.33	403.85
15	15	Noelani Patrick	sem.magna@google.com	2015-03-29 16:28:21 UTC	74.23	74.23
16	16	Eric Roth	lorem.sit@yahoo.net	2016-01-10 15:17:45 UTC	323.13	174.29
17	17	Bruce Gill	metus@aol.net	2015-10-07 04:51:06 UTC	224.09	263.24

The bottom of the results section shows 'Resultados por página: 50' and '1 - 50 de 5000'.

La consulta devolvió 5.000 registros, uno por usuario con al menos tres compras aceptadas. El resultado incluye su nombre completo, correo electrónico, fecha e importe de la tercera compra y el importe medio de sus tres primeras compras. Por ejemplo, el usuario 1, Zeus Gamble, realizó su tercera compra el 20 de diciembre de 2015 a las 18:06:59 UTC, por 161,33 €, mientras que la media de sus tres primeras compras fue de 239,13 €. Los resultados se presentan ordenados por identificador de usuario.

Nivell 3: Analytics Engineering (Arrays & Automatització)

Escenari:

Màrqueting necessita urgentment un Rànquing de Productes més Venuts.

Exercici 1: Desanidament i Aplanament de Dades (Unnesting)

Escenari:

Actualment, a la taula de transaccions, els productes venuts estan "amagats" dins d'un Array (una llista d'IDs). L'equip de vendes vol analitzar quins productes específics es venen més, però no poden fer-ho si les dades estan comprimides en una llista. Necessiten una Taula Detallada (Flat Table) on cada fila representi un producte venut, encara que això signifiqui que l'ID de la transacció es repeteixi.

Tasca:

Crea la taula `dim_transactions_flat` desnormalitzant la informació. Has d'"explotar" l'array de productes i creuar-lo amb el catàleg mestre per obtenir els noms i preus individuals.

Passos Tècnics:

- » 1. Utilitza `CROSS JOIN UNNEST(product_ids)` per transformar l'Array en files individuals.
 - » 2. Realitza un `JOIN` amb la taula products per obtenir el nom (name) i preu (price) de cada ítem.
-  *Nota: Ves amb compte amb els tipus de dades en fer el `JOIN` (Array de `INT64` vs `Product_ID STRING`).*

» 3. No agrupeu el resultat. Volem veure el desglossament línia per línia.

✓ Resultat Esperat (Visualització):

La teva taula final ha de veure's "plana", on una transacció amb 3 productes ocupa 3 files:

transaction_id	timestamp	total_ticket (Global)	product_sku	product_name	product_price
TXN1001	2025-12-08	155.97	SKU_A	Portàtil Bàsic	99.99
TXN1001	2025-12-08	155.97	SKU_B	Ratolí	12.99
TXN1001	2025-12-08	155.97	SKU_C	Funda	29.99
TXN1002	2025-12-07	45.50	SKU_D	Teclat	45.50

📁 Entrega:

- » Una captura de la Vista Prèvia de la teva taula on es vegi el DDL executada correctament.
- » Una captura de pantalla on es vegi l'esquema de la nova taula.

Se creará la tabla `sprint3_gold.dim_transactions_flat` a partir de las transacciones aceptadas de `transactions_clean`. Se desplegará el array `product_ids` mediante `CROSS JOIN UNNEST` y se incorporarán el nombre y el precio de cada producto desde `products_clean`. El resultado conservará una fila por elemento del array, sin agregaciones y manteniendo las fechas originales.

Se revisó el esquema de `products_clean` y se confirmó que `product_id` es de tipo `INTEGER`, compatible con los elementos `INT64` del array `product_ids` de `transactions_clean`, por lo que no fue necesario convertir tipos para realizar el `JOIN`. También se identificaron los campos `name` y `price`, utilizados para incorporar el nombre y el precio de cada producto.

The screenshot shows the Google Cloud BigQuery interface. The breadcrumb path is: sprint3-analytics-derian-palma / Datasets / sprint3_silver / Tables / products_clean. The table 'products_clean' is selected, and the 'Esquema' (Schema) tab is active. Below the breadcrumb, there are tabs for 'Esquema', 'Detalles', 'Vista previa', 'Estadísticas', 'Linaje', 'Perfil de datos', and 'Calidad de los datos'. A filter bar is present with the text 'Filtro Escribir el nombre o valor de la propiedad'. Below this is a table with 8 columns: 'Nombre del campo', 'Tipo', 'Modo', 'Descripción', 'Clave', 'Intercalación', 'Valor predeterminado', and 'Etiquetas de administración de datos'. The table contains 7 rows of data for the columns: product_id, name, warehouse_id, price, color, and weight.

Nombre del campo	Tipo	Modo	Descripción	Clave	Intercalación	Valor predeterminado	Etiquetas de administración de datos
product_id	INTEGER	NULLABLE	-	-	-	-	-
name	STRING	NULLABLE	-	-	-	-	-
warehouse_id	INTEGER	NULLABLE	-	-	-	-	-
price	FLOAT	NULLABLE	-	-	-	-	-
color	STRING	NULLABLE	-	-	-	-	-
weight	FLOAT	NULLABLE	-	-	-	-	-

At the bottom of the schema view, there are three buttons: 'Editar esquema', 'Ver políticas de acceso de fila', and 'Describir datos'.

Se creó una tabla plana mediante CROSS JOIN UNNEST(product_ids), incorporando el nombre y precio de cada producto desde products_clean. No fue necesario convertir los identificadores, ya que ambos utilizan tipos enteros compatibles. Se conservaron únicamente las transacciones aceptadas y no se aplicaron agregaciones. El importe total del ticket se repite en cada línea de una misma transacción, por lo que no debe sumarse directamente sobre esta tabla.

The screenshot shows the Google Cloud BigQuery console. The query editor is open, and a SQL query is being executed. The query is as follows:

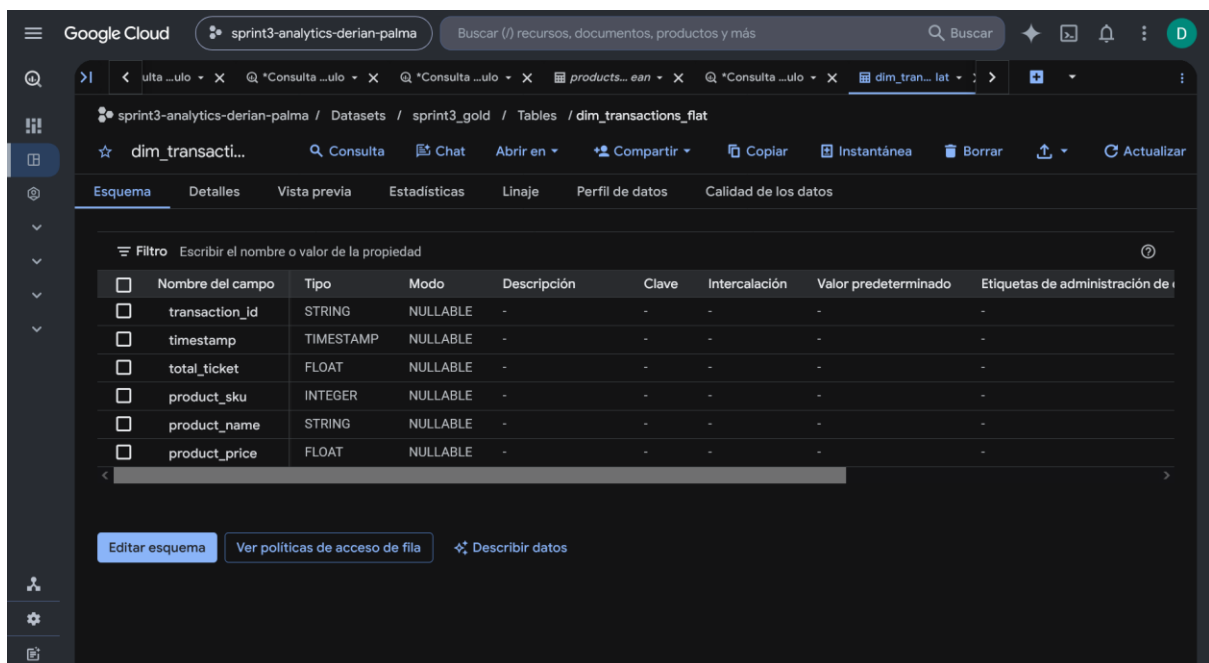
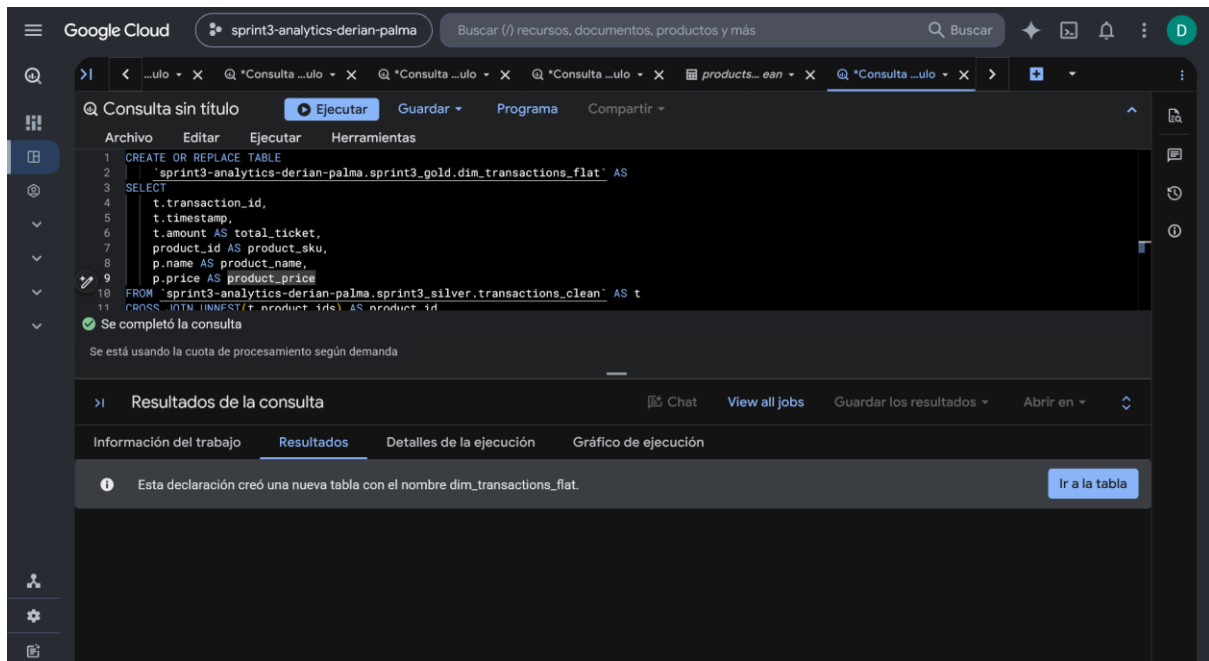
```

1 CREATE OR REPLACE TABLE
2   `sprint3-analytics-derian-palma.sprint3_gold.dim_transactions_flat` AS
3 SELECT
4   t.transaction_id,
5   t.timestamp,
6   t.amount AS total_ticket,
7   product_id AS product_sku,
8   p.name AS product_name,
9   p.price AS product_price
10  FROM `sprint3-analytics-derian-palma.sprint3_silver.transactions_clean` AS t
11  CROSS JOIN UNNEST(t.product_ids) AS product_id
12  JOIN `sprint3-analytics-derian-palma.sprint3_silver.products_clean` AS p
13    ON product_id = p.product_id
14  WHERE t.declined = 0;

```

At the bottom of the console, a status message indicates: 'Esta consulta procesará 7.85 MB cuando se ejecute.'

El validador estimó un procesamiento de 7,85 MB para crear la tabla. La ejecución finalizó correctamente y BigQuery confirmó la creación de dim_transactions_flat.



Se verificó que la tabla contiene las seis columnas previstas: transaction_id (STRING), timestamp (TIMESTAMP), total_ticket (FLOAT), product_sku (INTEGER), product_name (STRING) y product_price (FLOAT).

Fila	transaction_id	timestamp	total_ticket	product_sku	product_name	product_price
1	CDEC4BDC-EDFF-4015-B013-D...	2024-12-16 15:08:08 UTC	501.91	14	Direwolf	147.53
2	E3A984FF-7108-4805-BB0F-57...	2017-12-13 11:15:58 UTC	423.19	14	Direwolf	147.53
3	2320FOFA-6798-4803-80C4-2A...	2015-02-25 17:58:25 UTC	545.01	14	Direwolf	147.53
4	13529B83-C924-4875-8756-FB...	2020-04-15 21:55:26 UTC	505.29	14	Direwolf	147.53
5	88B1AB5C-9C83-4129-80C8-36...	2018-07-22 20:30:12 UTC	334.2	14	Direwolf	147.53
6	69DBF386-BBD2-406D-AE6B-5F...	2017-05-31 13:40:50 UTC	259.43	14	Direwolf	147.53
7	A66E5F77-A285-473D-82B6-98...	2015-09-24 13:46:40 UTC	398.9	14	Direwolf	147.53
8	33233F90-C8B5-4141-ACB1-EF...	2015-10-20 04:46:32 UTC	375.64	14	Direwolf	147.53
9	ECF6135F-9033-463D-89F6-164...	2017-06-19 17:38:41 UTC	370.14	14	Direwolf	147.53
10	D3ABE716-6A56-494C-BA2F-61...	2021-10-07 10:08:59 UTC	325.65	14	Direwolf	147.53
11	407A6849-CD02-4103-B703-17...	2024-05-15 06:56:41 UTC	530.13	14	Direwolf	147.53
12	475166CA-E683-414C-ABB0-F6...	2017-06-22 14:47:09 UTC	204.78	14	Direwolf	147.53
13	0D50316B-7074-4BA3-8CB7-B2...	2022-09-03 13:21:03 UTC	147.53	14	Direwolf	147.53
14	FDBB8F4E-2E2B-4236-A0E5-4F...	2016-05-26 23:06:45 UTC	464.59	14	Direwolf	147.53
15	2D078C1C-EAD1-402A-88D1-2D...	2020-03-19 01:25:07 UTC	488.97	14	Direwolf	147.53

La Vista previa muestra un total de 252.693 filas. En los registros visibles se observan las fechas originales de las transacciones y los datos del catálogo; por ejemplo, el producto Direwolf, con identificador 14 y precio 147,53.

```

1 SELECT *
2 FROM `sprint3-analytics-derian-palma.sprint3_gold.dim_transactions_flat`
3 ORDER BY transaction_id, product_sku
4 LIMIT 30;

```

Esta consulta procesará 20.91 MB cuando se ejecute.

Para visualizar juntas las líneas de cada transacción, se consultó la tabla ordenando por transaction_id y product_sku, con un límite de 30 filas. El validador estimó un procesamiento de 20,91 MB.

Google Cloud

sprint3-analytics-derian-palma

Buscar (/) recursos, documentos, productos y más

Buscar

En las primeras cuatro filas se observa una misma transacción del 28 de agosto de 2024, desglosada en los productos 16, 26, 87 y 97. La fecha y el importe global de 395,43 se repiten, mientras que cada fila identifica un producto distinto. Este ejemplo permite comprobar el resultado del desanidamiento.

Exercici 2: El Rànquing de Vendes (Agregació Simple)

Escenari:

Ara que has preparat la infraestructura i disposes de la taula `dim_transactions_flat` (on cada fila equival a un producte venut), generar informes analítics és trivial. Ja no necessites funcions complexes de desanidament per respondre preguntes bàsiques de negoci.

✂ Tasca:

Genera el **Top 5 de productes** més venuts en la història de la companyia.

🔧 Mètode:

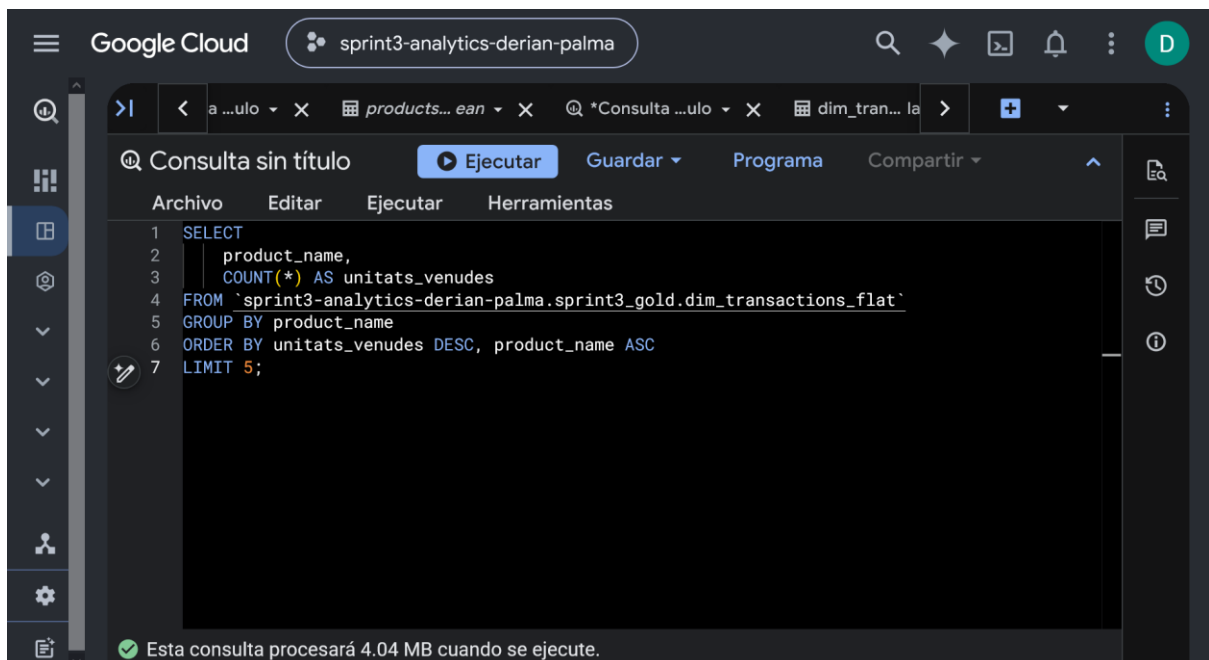
Consulta directament la teva nova taula desnormalitzada (`dim_transactions_flat`).

- » En ser la taula plana, simplement necessites una agrupació estàndard (GROUP BY) pel nom del producte i un compte (COUNT) de les files.

👉 Entrega:

- » El codi SQL que genera el rànquing.
- » Una captura del resultat amb els 5 productes guanyadors i quantes unitats s'han venut de cadascun.

Se obtendrá el Top 5 de productos más vendidos a partir de `dim_transactions_flat`. Se agruparán las filas por nombre del producto y se contará el número de apariciones de cada uno, considerando las transacciones aceptadas incluidas en la tabla.

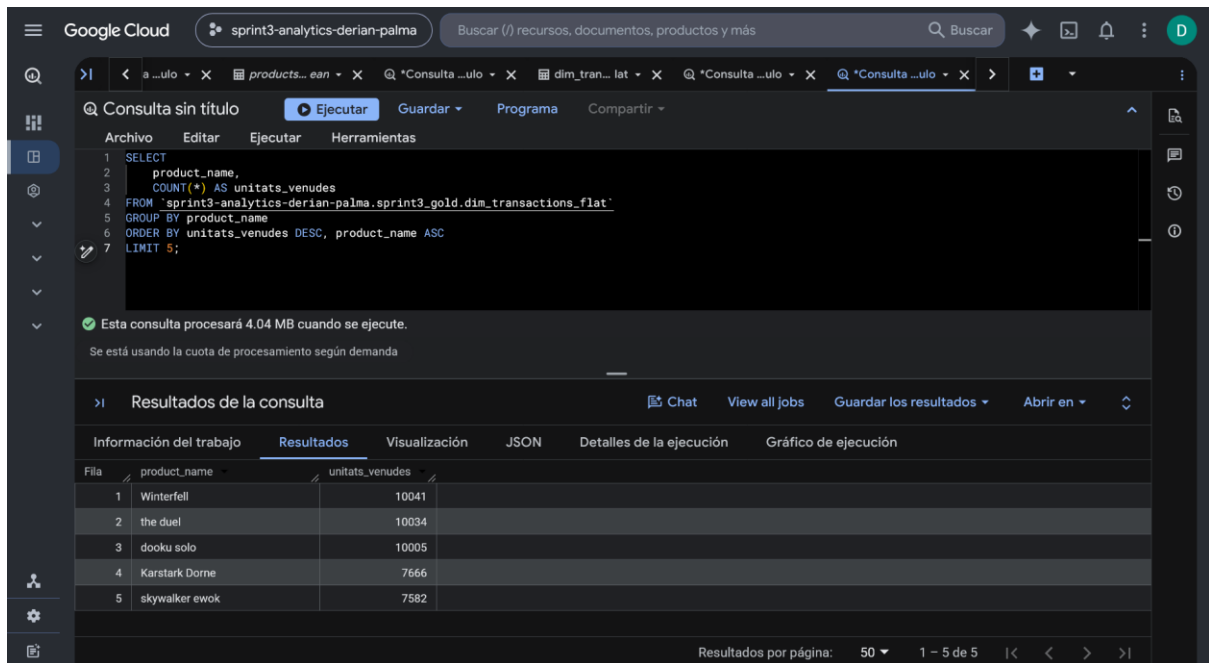


```
1 SELECT
2   product_name,
3   COUNT(*) AS unitats_venudes
4 FROM `sprint3-analytics-derian-palma.sprint3_gold.dim_transactions_flat`
5 GROUP BY product_name
6 ORDER BY unitats_venudes DESC, product_name ASC
7 LIMIT 5;
```

Esta consulta procesará 4.04 MB cuando se ejecute.

El validador confirmó que la consulta es válida y estimó un procesamiento de 4,04 MB para obtener el ranking de los cinco productos más vendidos.

La consulta utiliza `COUNT(*)` para contar las unidades vendidas por nombre de producto. Los resultados se ordenan de mayor a menor cantidad y se limitan a cinco registros. En caso de empate, se aplica el orden alfabético del nombre para obtener una selección reproducible.



The screenshot shows the Google Cloud BigQuery console interface. At the top, the Google Cloud logo and the project name 'sprint3-analytics-derian-palma' are visible. The main area displays a SQL query titled 'Consulta sin título'. The query is as follows:

```
1 SELECT
2   product_name,
3   COUNT(*) AS unitats_venudes
4 FROM `sprint3-analytics-derian-palma.sprint3_gold.dim_transactions_flat`
5 GROUP BY product_name
6 ORDER BY unitats_venudes DESC, product_name ASC
7 LIMIT 5;
```

Below the query, a status message indicates: 'Esta consulta procesará 4.04 MB cuando se ejecute. Se está usando la cuota de procesamiento según demanda'.

The results section, titled 'Resultados de la consulta', shows a table with the following data:

Fila	product_name	unitats_venudes
1	Winterfell	10041
2	the duel	10034
3	dooku solo	10005
4	Karstark Dorne	7666
5	skywalker ewok	7582

At the bottom right, the pagination information shows 'Resultados por página: 50' and '1 - 5 de 5'.

El ranking sitúa a Winterfell en primer lugar, con 10.041 unidades vendidas, seguido de the duel (10.034), dooku solo (10.005), Karstark Dorne (7.666) y skywalker ewok (7.582). Los recuentos corresponden a las transacciones aceptadas y se agrupan por nombre del producto.

Exercici 3: Automatització del Pipeline i Visualització

Escenari:

L'informe estàtic de l'exercici anterior ha agradat molt, però el Director de Màrqueting necessita professionalitzar-lo per a la presa de decisions diària. Té dos requeriments urgents:

- » **1. Lògica Centralitzada:** Volen veure el preu amb IVA (21%) desglossat. No obstant això, com que l'impost pot canviar per país o any, et prohibeixen posar el càlcul $\text{preu} * 1,21$ directament en el codi ("Hardcoded"). Has d'utilitzar una funció reutilitzable.
- » **2. Frescor de Dades:** El tauler ha d'estar actualitzat automàticament cada matí a les 07:00 AM, abans de la reunió diària de vendes.

Tasca:

Has d'evolucionar la teva taula `dim_transactions_flat` per incloure el càlcul d'impostos i automatitzar la seva regeneració.

Passos Tècnics:

- » **1. User Defined Functions (UDF):** Crea una funció SQL persistent anomenada `calculate_tax(amount)` que rebi un valor numèric i retorni el resultat aplicant el 21% d'impost.
- » **2. Integració i Orquestració:**
 - Modifica el codi de creació de la taula (de l'Exercici 1) perquè utilitzi la teva nova funció `calculate_tax` i generi una columna nova: `product_price_tax_inc`.
 - Configura una Scheduled Query a BigQuery perquè aquesta sentència (CTAS) s'executi automàticament cada dia a les 07:00 AM.
- » **3. Visualització (BI):** Connecta Looker Studio a la teva taula `dim_transactions_flat` i crea un Dashboard: "Monitor de Rendiment de Vendes"

Entrega:

- » El codi SQL de la UDF.
- » El codi SQL actualitzat de la creació de la taula.
- » Una captura de pantalla demostrant que la Scheduled Query està programada.
- » Un enllaç del gràfic a Looker Studio.

✚ Proposta de Dashboard:

"Monitor de Rendiment de Vendes"

Configura una pàgina amb 3 seccions clau:

» 1. Els "Big Numbers" (KPIs Generals):

A la part superior, col·loca tres targetes de resultats (Scorecards) per donar context immediat.

- **KPI 1: Ingressos Totals (Amb IVA)**
 - **Mètrica:** `SUM(product_price_tax_inc)`
 - *Per què:* Mostra el volum real de diners que entra.
- **Transaccions Úniques (Comandes)**
 - **Mètrica:** `COUNT_DISTINCT(transaction_id)`
- **Transaccions Úniques (Comandes)**
 - **Camp Calculat:** Crea un camp a Looker:

$$\frac{\text{SUM}(\text{product_price_tax_inc})}{\text{COUNT_DISTINCT}(\text{transaction_id})}$$

» 2. Evolució Temporal (Gràfic de Sèries Temporals)

Sota els KPIs, un gràfic de línia que ocupi tot l'ample.

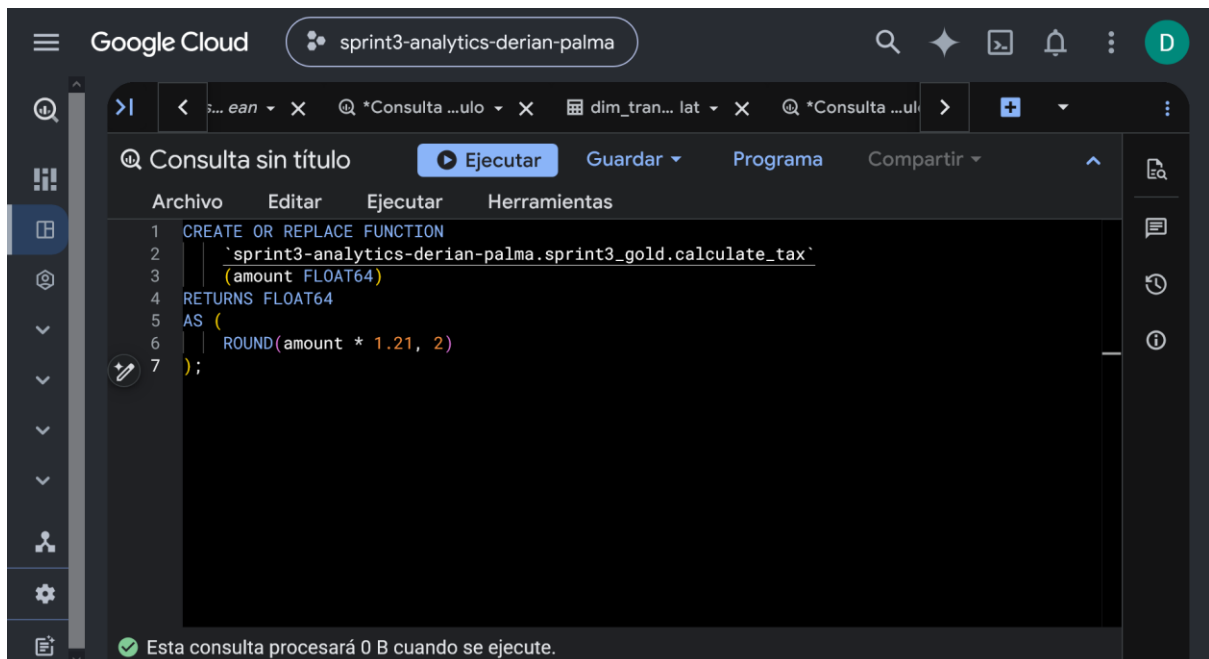
- **Dimensió:** timestamp (Desglossat per Data).
- **Mètrica:** `SUM(product_price_tax_inc)`.
- **Línia de Referència (Opcional):** Afegeix una mitjana constant o una línia de tendència.
- **Valor:** Permet a Màrqueting veure si les vendes pugen o baixen dia a dia, o si hi va haver un pic per alguna campanya específica.

» El "Top Productes" Enriquit (Taula amb Mapa de Calor)

Oblida't del gràfic de barres simple. Utilitza una Taula amb Barres incrustades o Mapa de Calor.

- **Dimensió:** `product_name`.
- **Mètrica 1:** `COUNT(*)` → Renombra-ho a "Unitats Venudes".
- **Mètrica 2:** `SUM(product_price_tax_inc)` → Renombra-ho a "Ingressos Totals".
- **Mètrica 3:** `SUM(product_price_tax_inc) - SUM(product_unit_price)` → Renombra-ho a "IVA Recaptat" (Això demostra el valor de la teva UDF).
- **Estil:** A la configuració d'estil de la taula, activa l'opció "Mostra número com a: Barra" o "Mapa de calor" per a la columna d'Ingressos.

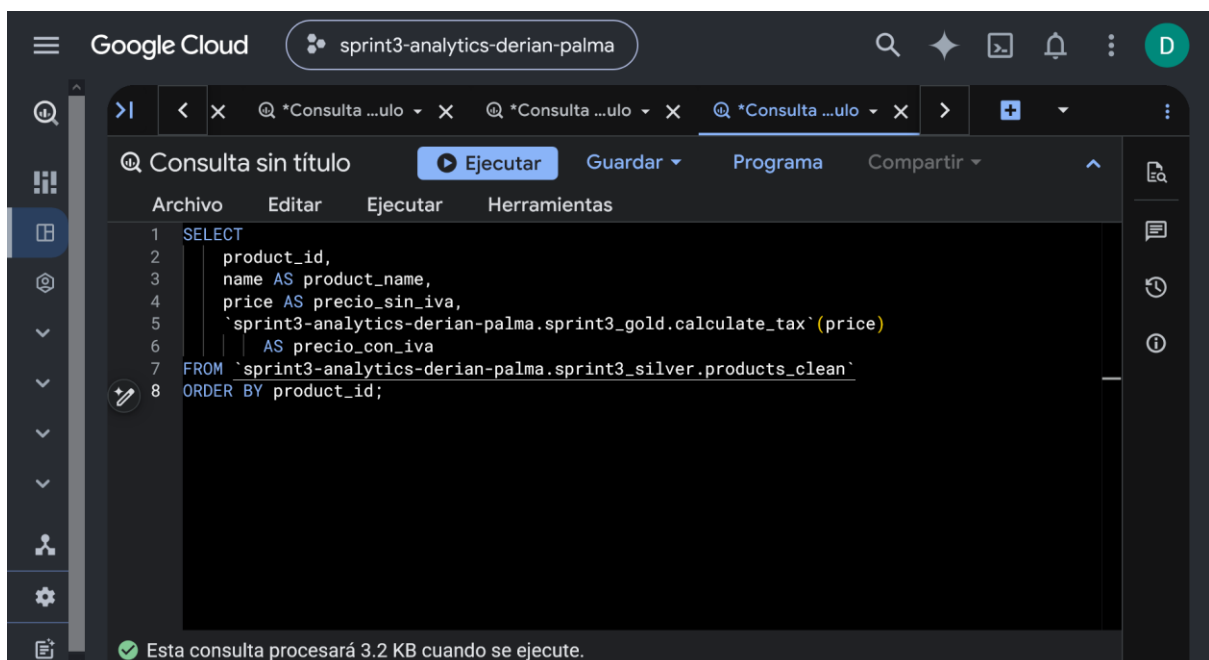
Se creará la función SQL persistente `calculate_tax` en el dataset `sprint3_gold` para centralizar el cálculo del precio con un 21 % de IVA. Esta función se reutilizará al generar la tabla plana, evitando repetir la fórmula en la consulta de creación.



```
1 CREATE OR REPLACE FUNCTION
2   `sprint3-analytics-derian-palma.sprint3_gold.calculate_tax`
3   (amount FLOAT64)
4   RETURNS FLOAT64
5   AS (
6     ROUND(amount * 1.21, 2)
7   );
```

Esta consulta procesará 0 B cuando se ejecute.

Se comprobará la función `calculate_tax` utilizando los precios reales de `products_clean`. La consulta mostrará, para cada producto, el precio original y el precio con IVA, permitiendo revisar su aplicación sobre importes con distintos valores y decimales.



```
1 SELECT
2   product_id,
3   name AS product_name,
4   price AS precio_sin_iva,
5   `sprint3-analytics-derian-palma.sprint3_gold.calculate_tax`(price)
6   AS precio_con_iva
7 FROM `sprint3-analytics-derian-palma.sprint3_silver.products_clean`
8 ORDER BY product_id;
```

Esta consulta procesará 3.2 KB cuando se ejecute.

Google Cloud sprint3-analytics-derian-palma

Buscar (f) recursos, documentos, productos y más

Consulta sin título

```

1 SELECT
2   product_id,
3   name AS product_name,
4   price AS precio_sin_iva,
5   sprint3-analytics-derian-palma.sprint3_gold.calculate_tax(price)
6   AS precio_con_iva
7 FROM `sprint3-analytics-derian-palma.sprint3_silver.products_clean`
8 ORDER BY product_id;

```

Se completó la consulta

Se está usando la cuota de procesamiento según demanda

Recientes 12:29 p.m.

Resultados de la consulta

Chat View all jobs Guardar los resultados Abrir en

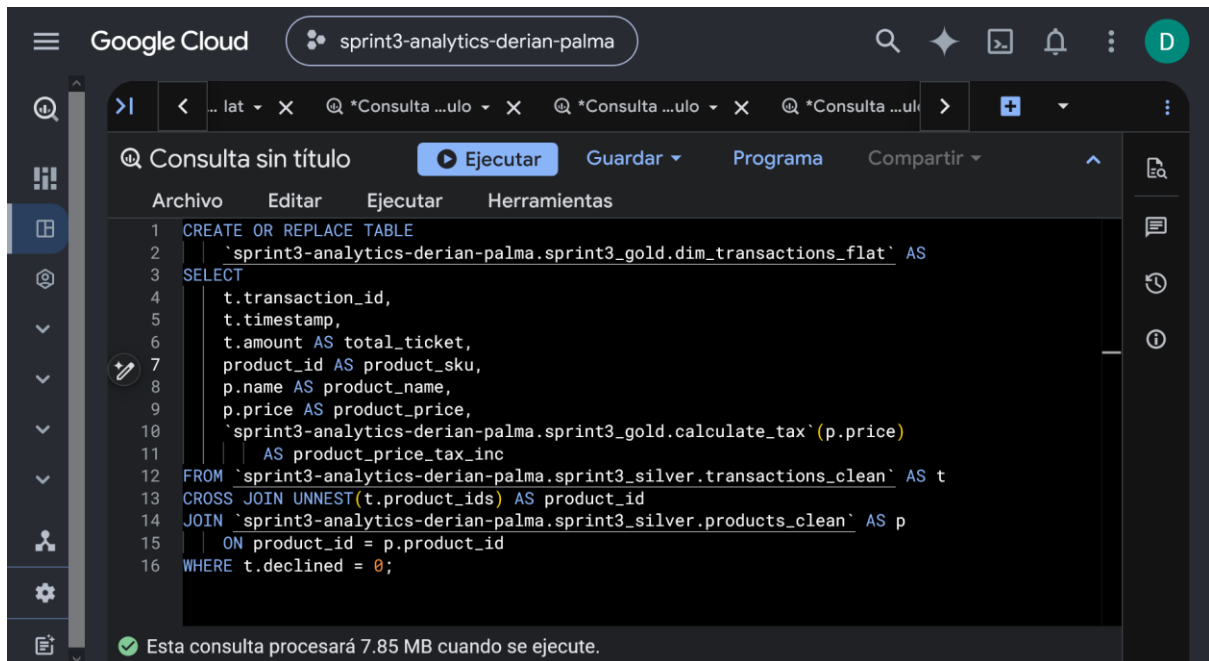
Información del trabajo Resultados Visualización JSON Detalles de la ejecución Gráfico de ejecución

Fila	product_id	product_name	precio_sin_iva	precio_con_iva
1	1	Direwolf Stannis	161.11	194.94
2	2	Tarly Stark	9.24	11.18
3	3	duel tourney Lannister	171.13	207.07
4	4	warden south duel	71.89	86.99
5	5	skywalker ewok	171.22	207.18

Resultados por página: 50 1 - 50 de 100

Se ejecutó la función `calculate_tax` sobre los precios de los 100 productos del catálogo. En los casos revisados, el resultado corresponde al precio original incrementado en un 21 % y redondeado a dos decimales. Por ejemplo, Direwolf Stannis pasa de 161,11 a 194,94 y Tarly Stark de 9,24 a 11,18. El validador estimó un procesamiento de 3,2 KB.

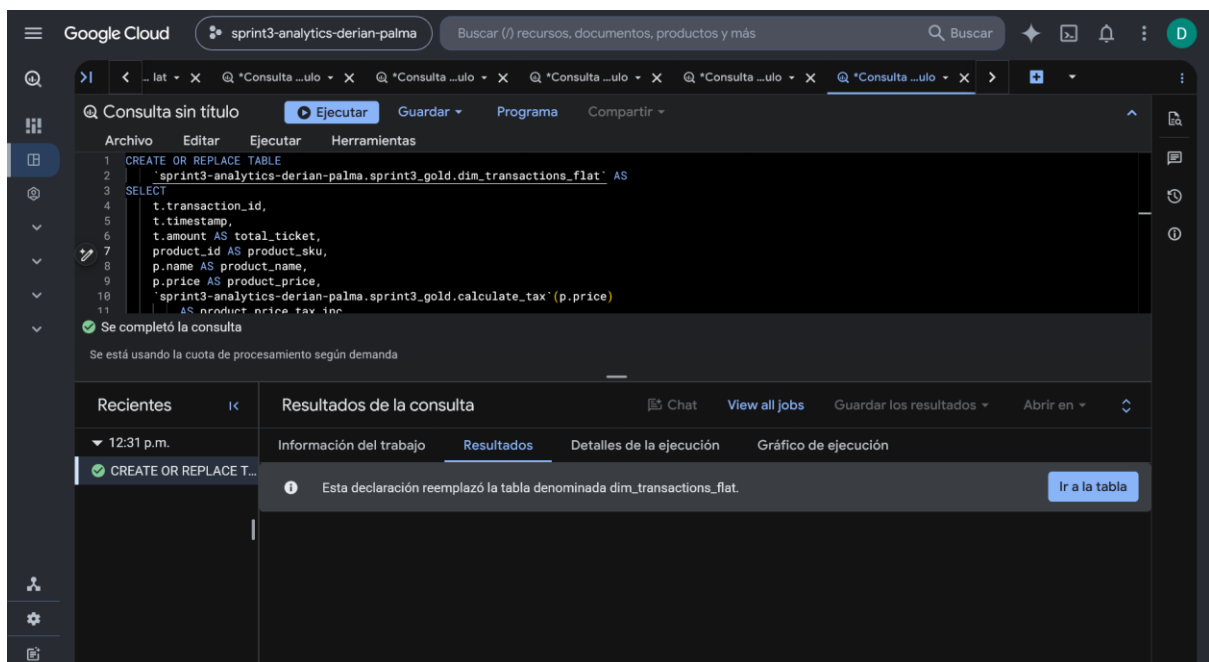
Se actualizará la tabla `dim_transactions_flat` para incorporar la columna `product_price_tax_inc`, calculada mediante la función persistente `calculate_tax`. Se mantendrá el detalle por producto y el filtro de transacciones aceptadas. La consulta aplica `calculate_tax` al precio de cada producto y almacena el resultado en la nueva columna. El cálculo del IVA queda centralizado en la función. `CREATE OR REPLACE TABLE` sustituye el contenido anterior por el resultado actualizado; si se modifica la función, será necesario volver a ejecutar esta consulta para actualizar los precios almacenados.



The screenshot shows the Google Cloud BigQuery console interface. At the top, the project name 'sprint3-analytics-derian-palma' is visible. The main area contains a SQL query editor with the following code:

```
1 CREATE OR REPLACE TABLE
2   `sprint3-analytics-derian-palma.sprint3_gold.dim_transactions_flat` AS
3 SELECT
4   t.transaction_id,
5   t.timestamp,
6   t.amount AS total_ticket,
7   product_id AS product_sku,
8   p.name AS product_name,
9   p.price AS product_price,
10  `sprint3-analytics-derian-palma.sprint3_gold.calculate_tax`(p.price)
11    AS product_price_tax_inc
12 FROM `sprint3-analytics-derian-palma.sprint3_silver.transactions_clean` AS t
13 CROSS JOIN UNNEST(t.product_ids) AS product_id
14 JOIN `sprint3-analytics-derian-palma.sprint3_silver.products_clean` AS p
15   ON product_id = p.product_id
16 WHERE t.declined = 0;
```

Below the query editor, a status message indicates: 'Esta consulta procesará 7.85 MB cuando se ejecute.'



La consulta se ejecutó correctamente y BigQuery confirmó el reemplazo de `dim_transactions_flat`, incorporando el cálculo del precio con IVA mediante `calculate_tax`. El validador estimó un procesamiento de 7,85 MB.

Google Cloud | sprint3-analytics-derian-palma | Buscar (f) recursos, documentos, productos y más

sprint3-analytics-derian-palma / Datasets / sprint3_gold / Tables / dim_transactions_flat

☆ dim_transacti... Consulta Chat Abrir en Compartir Copiar Instantánea Borrar Actualizar

Esquema Detalles Vista previa Estadísticas Linaje Perfil de datos Calidad de los datos

Filtro Escribir el nombre o valor de la propiedad

Nombre del campo	Tipo	Modo	Descripción	Clave	Intercalación	Valor predeterminado	Etiquetas de administrac
transaction_id	STRING	NULLABLE	-	-	-	-	-
timestamp	TIMESTAMP	NULLABLE	-	-	-	-	-
total_ticket	FLOAT	NULLABLE	-	-	-	-	-
product_sku	INTEGER	NULLABLE	-	-	-	-	-
product_name	STRING	NULLABLE	-	-	-	-	-
product_price	FLOAT	NULLABLE	-	-	-	-	-
product_price_tax_inc	FLOAT	NULLABLE	-	-	-	-	-

Editar esquema Ver políticas de acceso de fila Describir datos

Se verificó en el esquema la incorporación de `product_price_tax_inc`, de tipo `FLOAT`, junto con las seis columnas anteriores.

Google Cloud | sprint3-analytics-derian-palma | Buscar (f) recursos, documentos, productos y más

sprint3-analytics-derian-palma / Datasets / sprint3_gold / Tables / dim_transactions_flat

☆ dim_transacti... Consulta Chat Abrir en Compartir Copiar Instantánea Borrar Actualizar

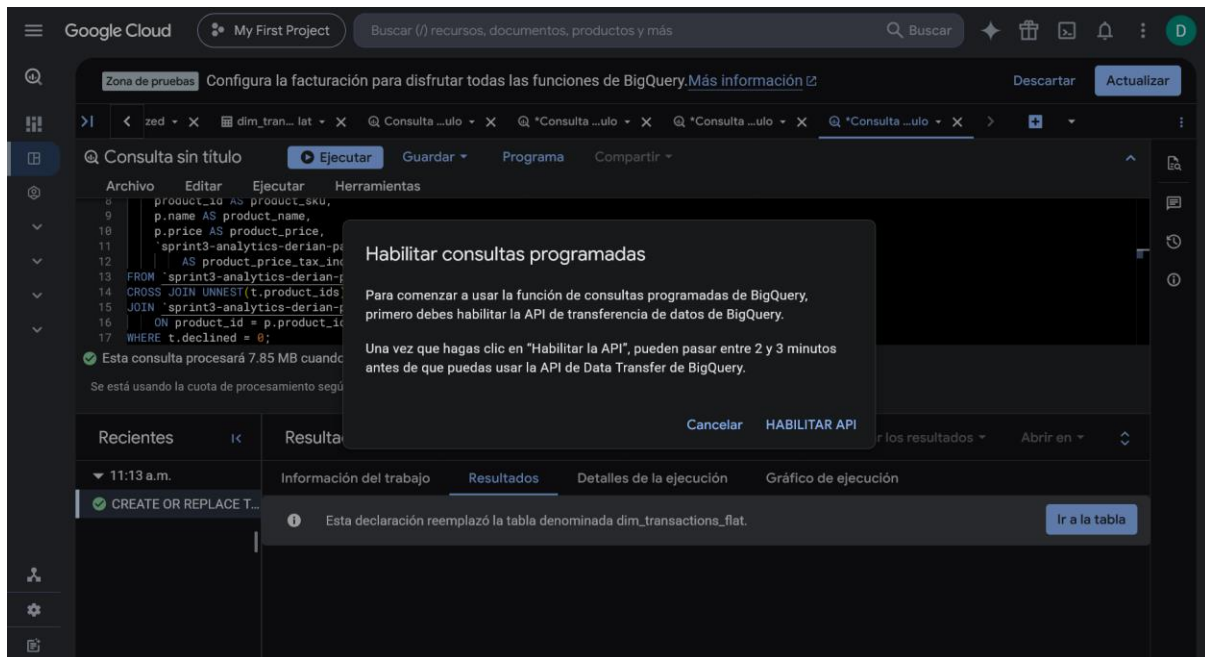
Esquema Detalles Vista previa Estadísticas Linaje Perfil de datos Calidad de los datos

Fila	transaction_id	timestamp	total_ticket	product_sku	product_name	product_price	product_pric...
1	EDFA0FAE-053A-41AF-9591-43...	2023-01-09 04:52:05 UTC	243.75	14	Direwolf	147.53	178.51
2	7E0E78CD-6F72-4794-AD3-AF...	2016-12-02 23:58:38 UTC	407.83	14	Direwolf	147.53	178.51
3	9C00045D-0DED-4446-B86C-1B...	2015-06-01 23:27:39 UTC	434.56	14	Direwolf	147.53	178.51
4	0A654DB4-1663-43C2-9E58-34...	2017-12-10 18:58:26 UTC	276.01	14	Direwolf	147.53	178.51
5	6CSA8AD5-1B67-46C1-BE0C-E7...	2015-12-24 14:16:09 UTC	205.25	14	Direwolf	147.53	178.51
6	57523773-DC5F-465A-8EFE-4E...	2016-02-16 14:55:06 UTC	362.62	14	Direwolf	147.53	178.51
7	B99F2C9E-94B4-4C3D-9F48-00...	2017-07-31 23:16:09 UTC	447.49	14	Direwolf	147.53	178.51
8	FC965A47-7476-4A10-B736-16...	2019-03-14 16:48:59 UTC	238.85	14	Direwolf	147.53	178.51
9	0509BEE8-52C1-4919-A84A-6E...	2018-06-24 04:51:57 UTC	147.53	14	Direwolf	147.53	178.51
10	4D40C0F5-A4EF-4CBE-AABE-05...	2015-11-12 21:04:09 UTC	281.65	14	Direwolf	147.53	178.51
11	8DB8FA93-9B86-4C83-B75B-59...	2024-02-10 08:06:22 UTC	466.15	14	Direwolf	147.53	178.51
12	2F066F9F-02EB-4B50-B487-FAE...	2021-12-10 15:18:55 UTC	395.01	14	Direwolf	147.53	178.51
13	7ADDCBC8-C38B-4DA1-8729-0...	2023-08-27 20:30:56 UTC	405.95	14	Direwolf	147.53	178.51
14	04A3DB3B-0D48-49B4-8BC1-76...	2015-09-05 23:20:05 UTC	308.64	14	Direwolf	147.53	178.51
15	5F8FDC04-67D7-44E9-A6B4-ED...	2019-08-09 00:48:42 UTC	310.38	14	Direwolf	147.53	178.51

Resultados por página: 50 1 - 50 de 252693

Se comprobó en la Vista previa la incorporación del precio con IVA. Para el producto Direwolf, el precio original de 147,53 se transforma en 178,51 mediante la función `calculate_tax`. La tabla conserva las 252.693 filas del detalle de productos vendidos.

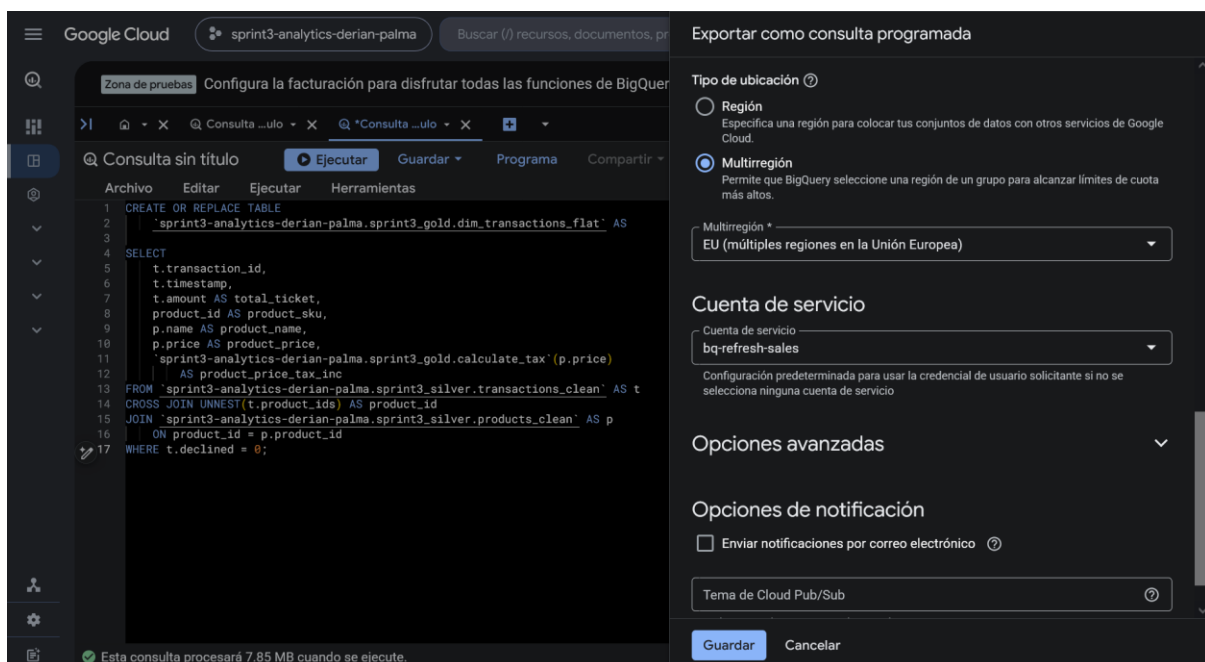
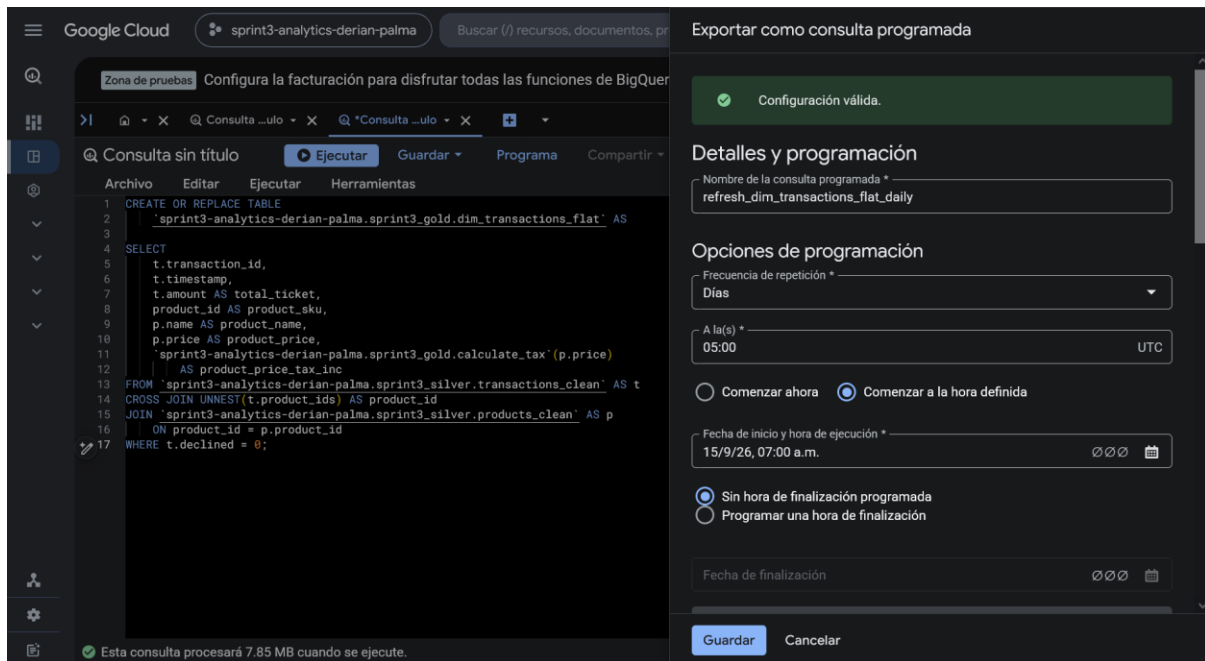
Se iniciará la configuración de una consulta programada para regenerar diariamente `dim_transactions_flat` a las 07:00, utilizando la sentencia de creación que incorpora el precio con IVA. Se comprobará la disponibilidad de esta funcionalidad en el entorno Sandbox.



Al iniciar la configuración de la consulta programada, BigQuery solicitó habilitar la API de transferencia de datos como requisito previo. En este punto, la programación diaria todavía no se ha configurado.

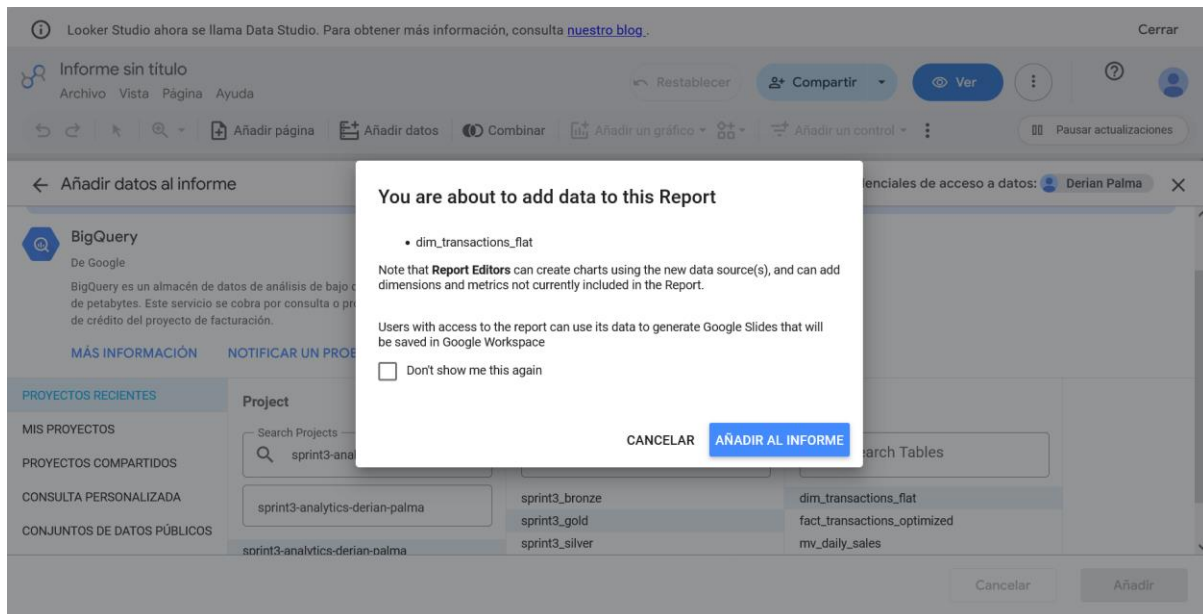
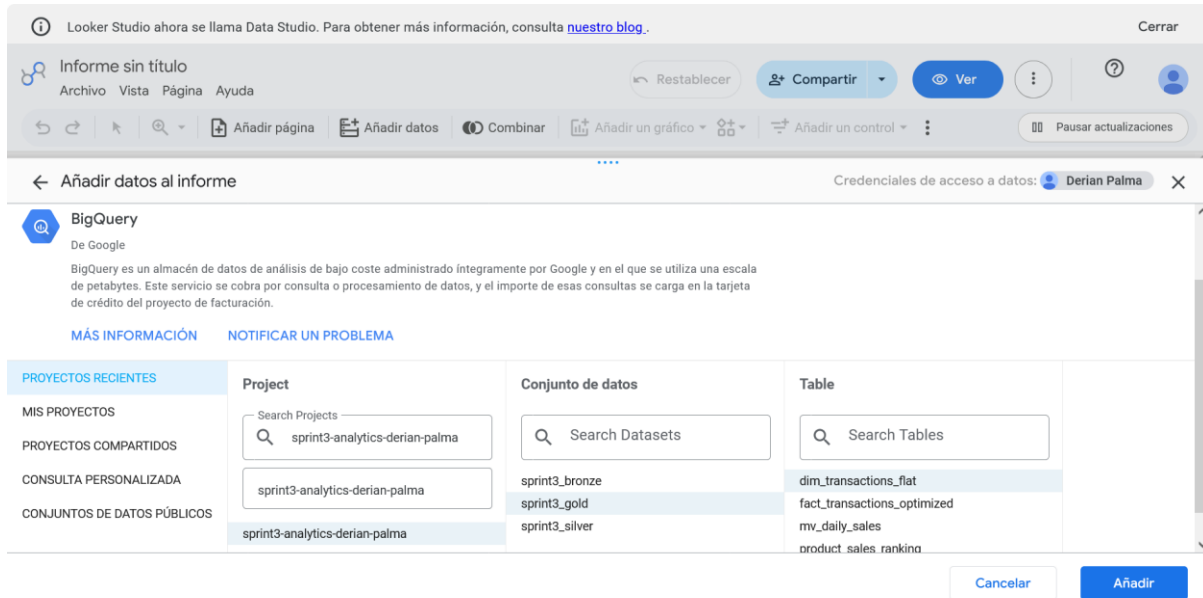
Se preparó la consulta programada `refresh_dim_transactions_flat_daily` para regenerar la tabla mediante la sentencia `CREATE OR REPLACE TABLE` que incorpora la función `calculate_tax`. Se estableció una frecuencia diaria a las 05:00 UTC, equivalentes a las 07:00 de Barcelona en horario de verano, sin fecha de finalización. Para mantener las 07:00 locales en invierno, será necesario ajustar la hora a las 06:00 UTC.

Se seleccionaron la multirregión EU y la cuenta de servicio `bq-refresh-sales` para ejecutar la consulta. Se dejó desmarcada la opción de establecer una tabla de destino, ya que la propia sentencia SQL especifica la tabla que debe regenerarse.

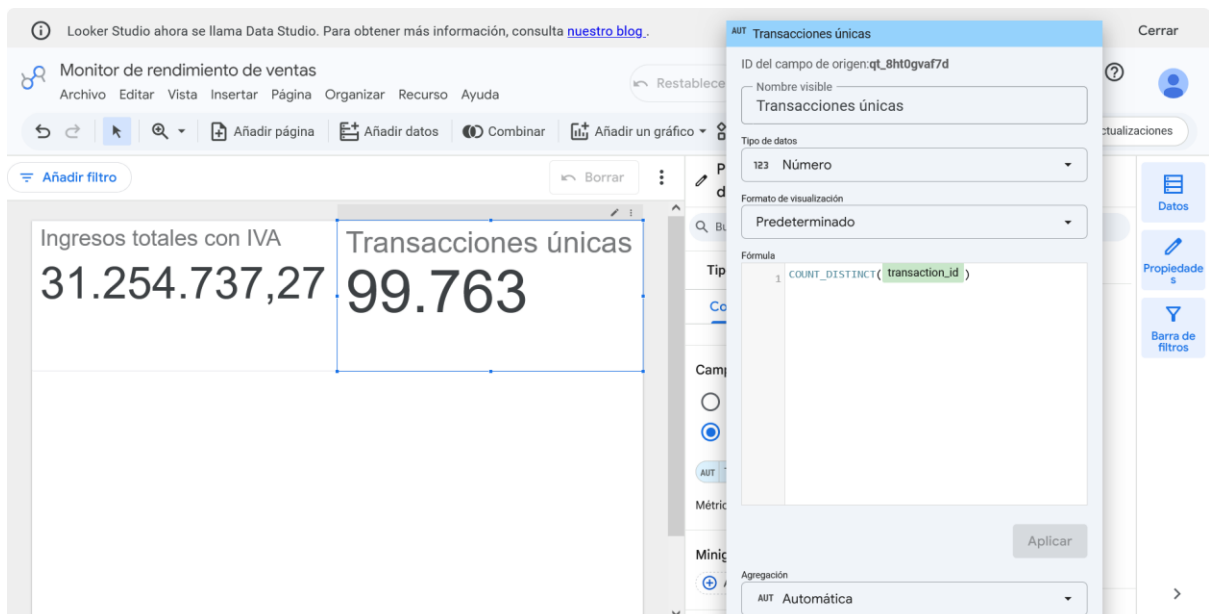
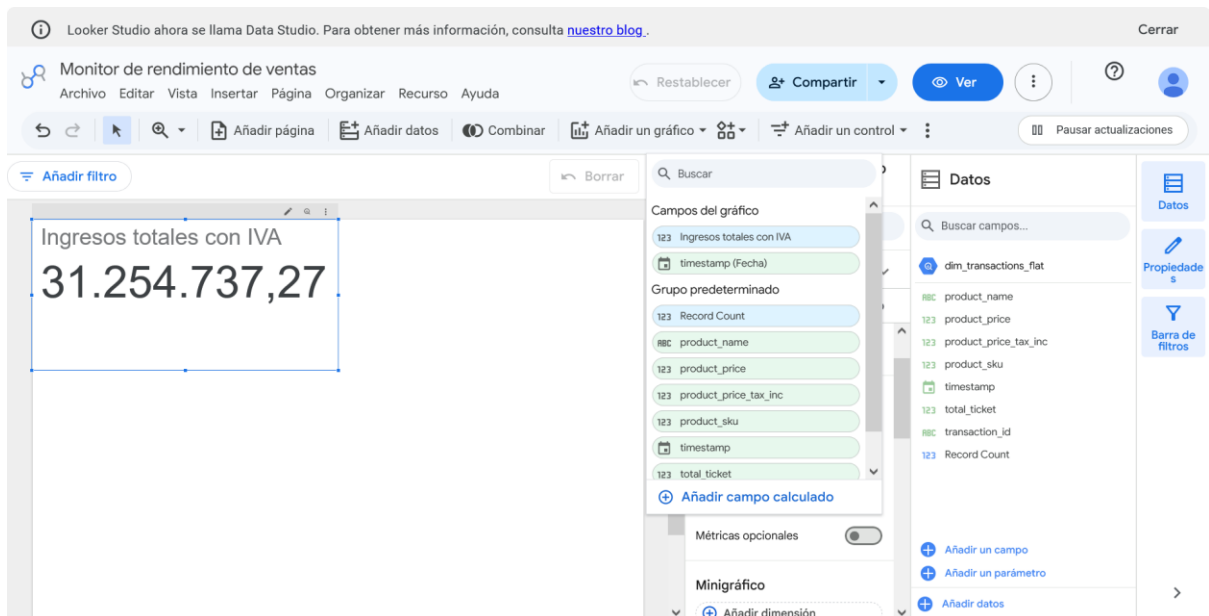


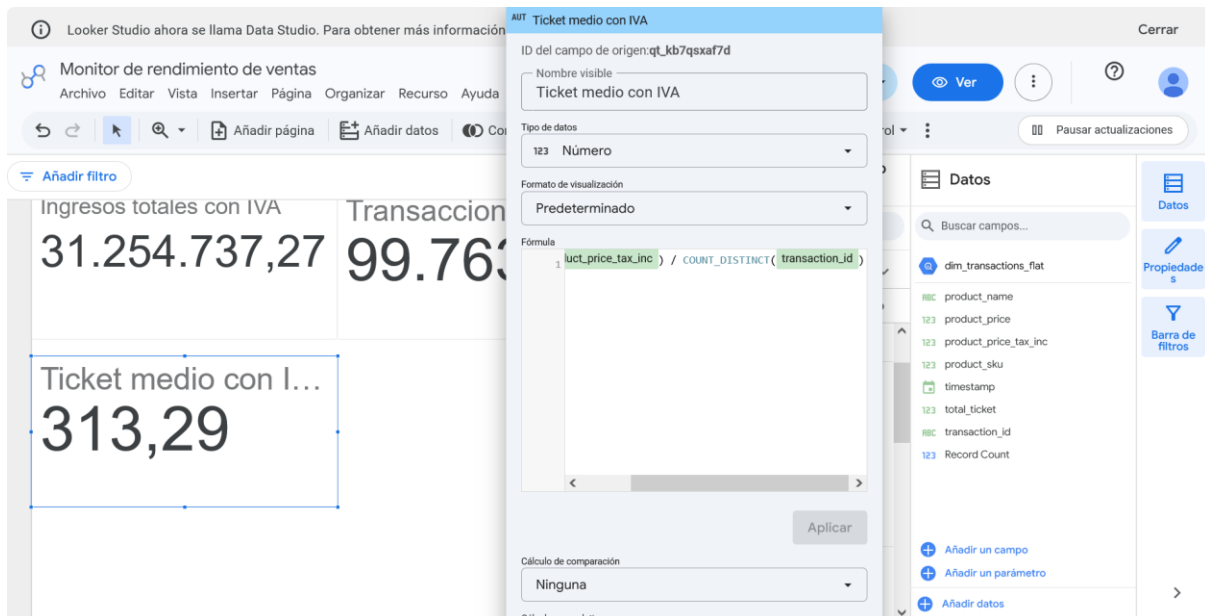
Para facilitar el análisis de las ventas, se creará el dashboard «Monitor de rendimiento de ventas» en Looker Studio, conectado a la tabla `dim_transactions_flat` de BigQuery. El informe incluirá tres indicadores generales —ingresos con IVA, transacciones únicas y ticket medio—, un gráfico de evolución diaria y una tabla de productos con unidades vendidas, ingresos e IVA recaudado. Se configurarán los campos calculados y el formato visual para presentar la información de forma clara y facilitar su interpretación.

Se conectó el informe «Monitor de rendimiento de ventas» a la tabla `sprint3_gold.dim_transactions_flat` mediante el conector de BigQuery. Esta fuente contiene el detalle de las transacciones aceptadas por producto y el precio con IVA calculado mediante la función `calculate_tax`.

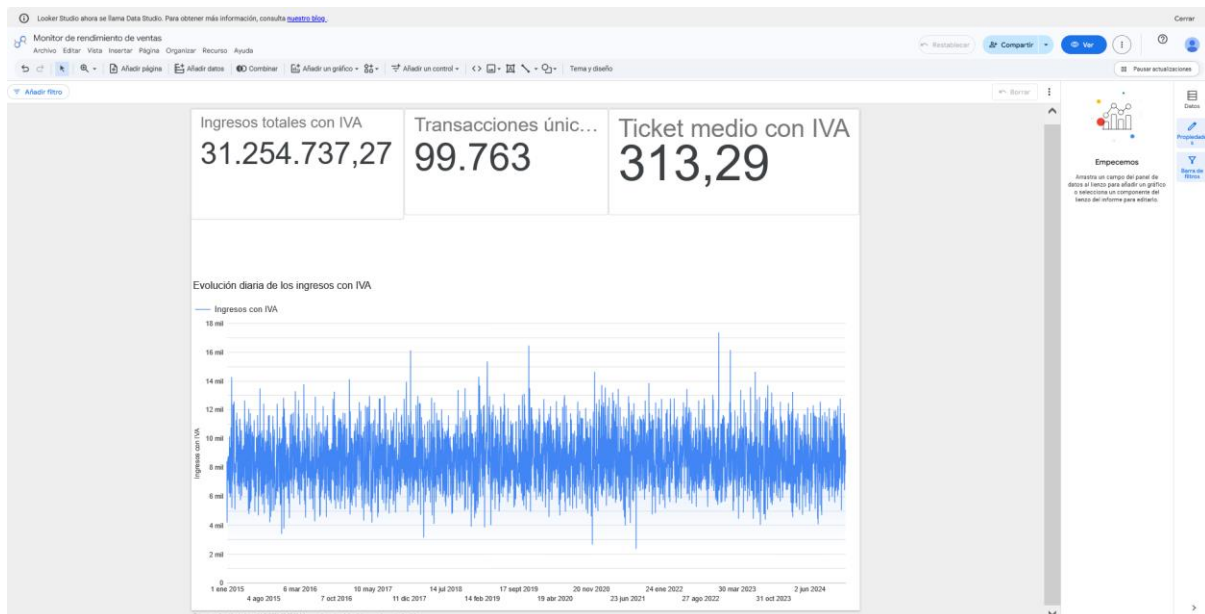


Se configuraron tres indicadores: ingresos totales con IVA mediante `SUM(product_price_tax_inc)`, transacciones únicas mediante `COUNT_DISTINCT(transaction_id)` y ticket medio con IVA mediante la división de ambas métricas. El recuento distinto evita contabilizar varias veces una transacción desglosada en diferentes productos. Los importes se muestran con dos decimales y el número de transacciones sin decimales.





Se creó una serie temporal con timestamp como dimensión de fecha y la suma de `product_price_tax_inc` como métrica. El gráfico representa la evolución diaria de los ingresos con IVA durante el periodo histórico disponible, permitiendo observar fluctuaciones y picos de ventas.



Se configuró una tabla agrupada por `product_name`, con el recuento de filas como unidades vendidas, la suma de los precios con IVA como ingresos y la diferencia $\text{SUM}(\text{product_price_tax_inc}) - \text{SUM}(\text{product_price})$ como IVA recaudado. Se ordenó por unidades vendidas de mayor a menor y se aplicó un mapa de calor a los ingresos. Los importes se presentan con dos decimales.

Looker Studio ahora se llama Data Studio. Para obtener más información, consulta [nuestro blog](#).

Monitor de rendimiento de ventas

Archivo Editar Vista Insertar Página Organizar Recurso Ayuda

Restablecer Compartir Ver Pausar actualizaciones

Añadir filtro

Producto Unidades... Ingr... IVA rec...

	Producto	Unidades...	Ingr...	IVA rec...
1.	Winterfell	10.041	1.052...	182.581,45
2.	the duel	10.034	1.270...	220.521,82
3.	dooku solo	10.005	693.8...	120.416,57
4.	Karstark Dorne	7.666	849.2...	147.390,96
5.	skywalker ewok	7.582	1.533...	266.213,2
6.	Karstark warden	7.539	1.243...	215.797,48
7.	Direwolf Stannis	7.421	845.9...	146.816,01
8.	duel warden	5.158	890.0...	154.467,77
9.	duel Direwolf	5.151	865.0	150.151,77

Propiedades del Tabla

Buscar configuración...

Producto

ID del campo de origen: product_name

Nombre visible: Producto

Campo de origen: product_name

Tipo de datos: Texto

Añadir acción para extraer detalles

Añadir dimensión

Desglosar información

Métrica

Unidades vendidas

Looker Studio ahora se llama Data Studio. Para obtener más información, consulta [nuestro blog](#).

Monitor de rendimiento de ventas

Archivo Editar Vista Insertar Página Organizar Recurso Ayuda

Restablecer Compartir Ver Pausar actualizaciones

Añadir filtro

product_name Unidades... Ingr... IVA rec...

	product_name	Unidades...	Ingr...	IVA rec...
1.	Winterfell	10.041	1.052...	182.581,45
2.	the duel	10.034	1.270...	220.521,82
3.	dooku solo	10.005	693.8...	120.416,57
4.	Karstark Dorne	7.666	849.2...	147.390,96
5.	skywalker ewok	7.582	1.533...	266.213,2
6.	Karstark warden	7.539	1.243...	215.797,48
7.	Direwolf Stannis	7.421	845.9...	146.816,01
8.	duel warden	5.158	890.0...	154.467,77
9.	duel Direwolf	5.151	865.0	150.151,77

Unidades vendidas

Nombre visible: Unidades vendidas

Campo de origen: Record Count

Tipo de datos: Número

Formato de visualización: Predeterminado

Agregación: Automática

Cálculo de comparación: Ninguna

Cálculo acumulativo: Ninguna

Añadir métrica

Métricas opcionales

Controles deslizantes de métricas

Filtrar

Filtros de este gráfico

Añadir filtro

The screenshot shows the Google Data Studio interface. A table visualization is displayed with the following data:

product_name	Record Count	Ingresos totales con IVA	IVA recaudado
Winterfell	10.041	1.052.000	182.581,45
the duel	10.034	1.270.000	220.521,82
dooku solo	10.005	693.800	120.416,57
Karstark Dorne	7.666	849.200	147.390,96
skywalker ewok	7.582	1.533.000	266.213,2
Karstark warden	7.539	1.243.000	215.797,48
Direwolf Stannis	7.421	845.900	146.816,01
duel warden	5.158	890.000	154.467,77
duel Direwolf	5.151	865.000	150.151,77

The 'Ingresos totales con IVA' configuration panel is open, showing the following settings:

- ID del campo de origen: `product_price_tax_inc`
- Nombre visible: Ingresos totales con IVA
- Campo de origen: `product_price_tax_inc`
- Tipo de datos: 123 Número
- Formato de visualización: Predeterminado
- Agregación: SUM Total
- Cálculo de comparación: Ninguna
- Cálculo acumulativo: Ninguna

The 'Propiedades del Tabla' panel on the right shows the following settings:

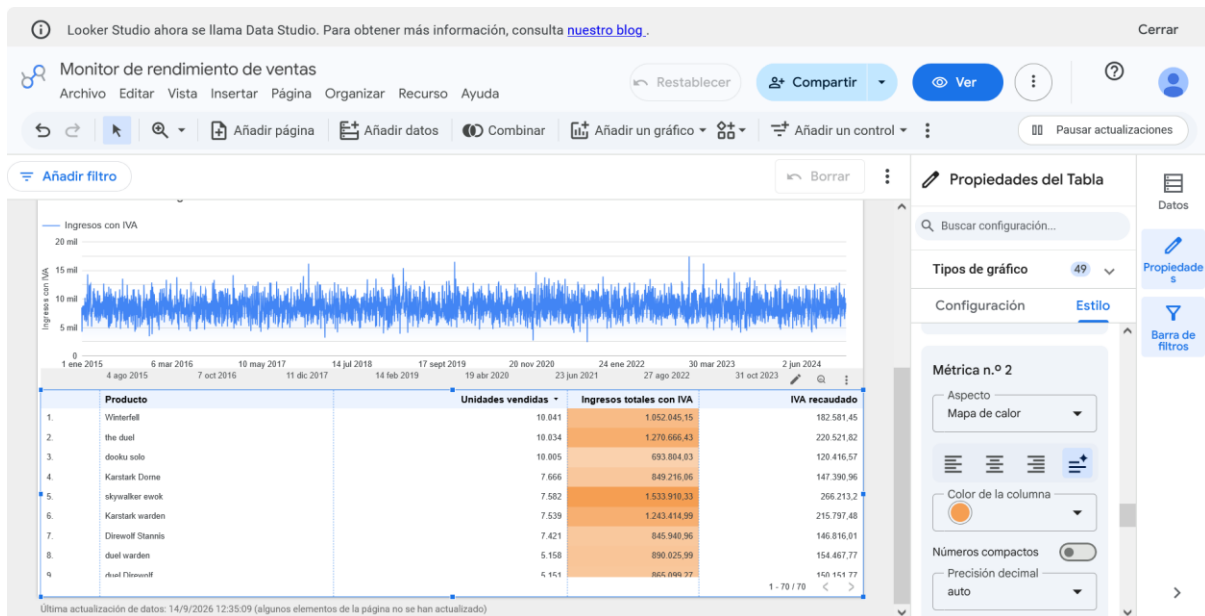
- Tipo de gráfico: 49
- Configuración: Record Count
- Estilo: SUM Ingresos totales con IVA
- Métricas opcionales: ☐
- Controles deslizando de métricas: ☐
- Filtrar: Filtros de este gráfico
- Añadir filtro:

The screenshot shows the Google Data Studio interface. A table visualization is displayed with the following data:

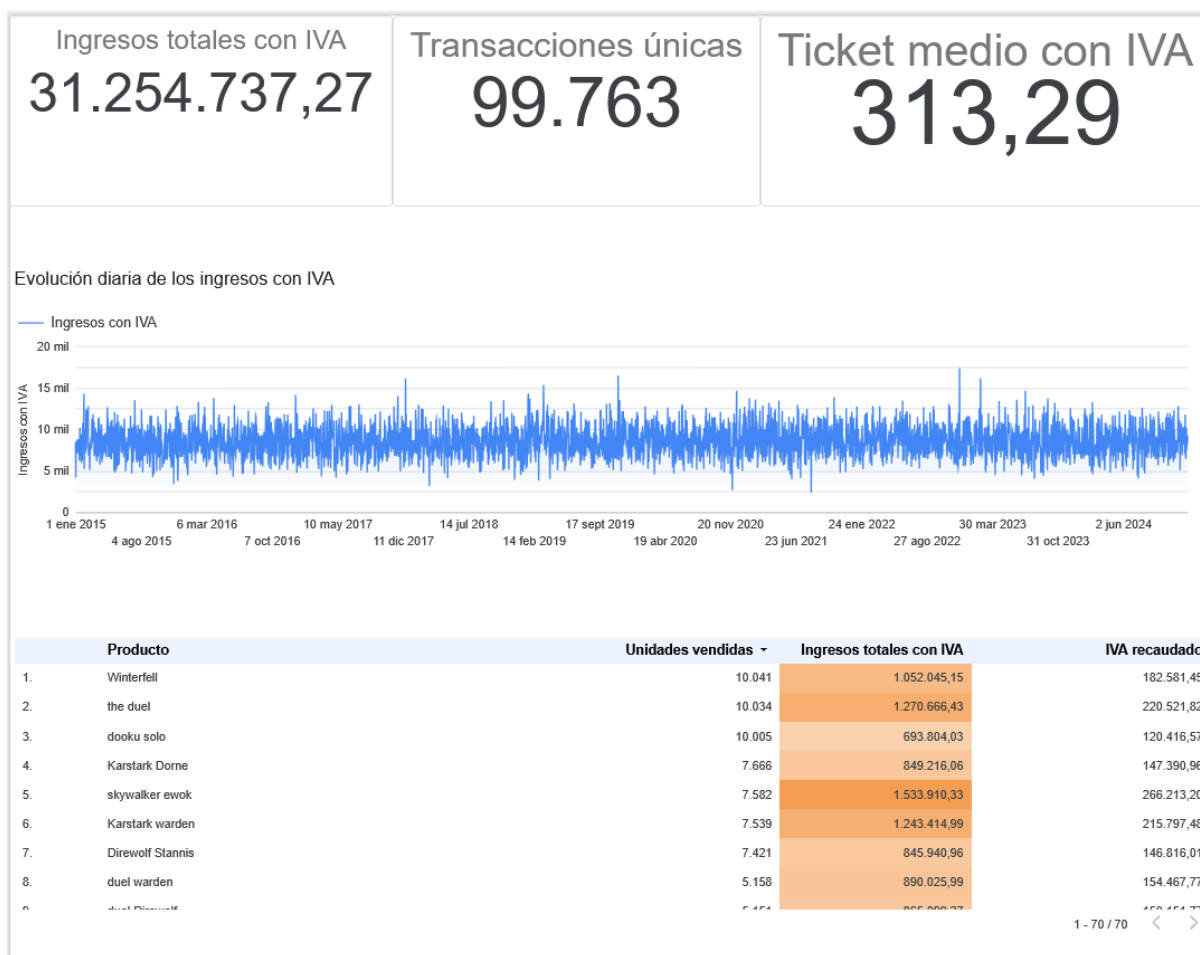
product_name	Record Count	prod...	IVA rec...
Winterfell	10.041	1.052.000	182.581,45
the duel	10.034	1.270.000	220.521,82
dooku solo	10.005	693.800	120.416,57
Karstark Dorne	7.666	849.200	147.390,96
skywalker ewok	7.582	1.533.000	266.213,2
Karstark warden	7.539	1.243.000	215.797,48
Direwolf Stannis	7.421	845.900	146.816,01
duel warden	5.158	890.000	154.467,77
duel Direwolf	5.151	865.000	150.151,77

The 'IVA recaudado' configuration panel is open, showing the following settings:

- ID del campo de origen: `qt_slpp09af7d`
- Nombre visible: IVA recaudado
- Tipo de datos: 123 Número
- Formato de visualización: Predeterminado
- Fórmula: `SUM( product_price_tax_inc ) - SUM( product_price )`
- Agregación: AUT Automática
- Cálculo de comparación:



El dashboard integra los indicadores generales, la evolución diaria de los ingresos con IVA y el detalle por producto. Para el periodo mostrado, presenta ingresos con IVA de 31.254.737,27, un total de 99.763 transacciones únicas y un ticket medio de 313,29. La tabla permite comparar las unidades vendidas, los ingresos y el IVA recaudado, con un mapa de calor que destaca las diferencias de ingresos entre productos. Las métricas monetarias se calculan a partir de los precios de los productos, evitando sumar el importe global del ticket que se repite en cada línea.



Enlace al dashboard:

<https://datastudio.google.com/reporting/a1082a97-014f-46f8-996e-2025ca5e0aa7>